Thimza: Practical Lowstate Threshold Raccoon Signatures at Low Bandwidth

***Abstract*—Transition to post-quantum is accelerating, and threshold digital signatures are not an exception. A (t, n) threshold signature lets any t of n parties jointly sign a message, while $t-1$ corrupt parties cannot forge. Because lattice signatures are promising, e.g. Quorus [USENIX'26], the most efficient lattice threshold signatures are based on the Raccoon [CRYPTO'24] family, e.g., Ringtail [S&P'25]. However, this family has three problems. Firstly, in Threshold Raccoon [EUROCRYPT'24], the signers must save all of the session identifiers they have replied to. Secondly, this family requires authenticated channels because, in a first round where the signer set is known or large preprocessed messages are used, a signer sends 28 to 852 KB at $t = 1024$. Thirdly, their security has been proven only against classical adversaries directly. Besides these, as the 6G technology gets closer, Non-terrestrial Networks (NTN) will be a core part of it, as 3GPP has formally added NTN to security study, and IRTF activated SPACE RG. However, post-quantum systems have not been publicly tested on NTN yet, and their precise cost on real-world satellites is unknown.**

In this work, we solve all these problems. We prove that because a signer whose response is linear in its key share leaks the share when its last round is replayed, some state is unavoidable. Then, we present Thimza[1]**, a three round threshold signature whose signatures are Raccoon signatures. In our** Thimza**, (i) a signer only need to keep one token for each open session, (ii) channels do not require authentication and the first round is independent of the signer set, and (iii) at the 128 bit level and $t = 1024$ a signer sends 16.4 KiB and signs 2.2 times faster than in Threshold Raccoon because signers open only the high order bits of their commitments. In addition,** Thimza **achieves the strongest threshold unforgeability security notion (TS-UF-4) in the quantum random oracle model. Moreover, we provide machine-checked proofs in `EasyCrypt`, with the quantum random oracle modelled as in `EasyPQC`. Also, for the first time, we practically deploy and measure the NTN applicability of our** Thimza **post-quantum scheme on satellites via `LEOScope` over an operational Starlink link.**

1. Introduction

In a (t, n) threshold signature [1], [2], a signing key is shared among n parties so that any t of them can jointly sign a message, while an adversary that corrupts $t-1$ parties cannot forge. Threshold Schnorr signatures such as FROST [2] are deployed in custody systems and blockchains, and their security is well understood [3]. Their lattice counterparts are needed against quantum adversaries, and the most efficient ones are based on Raccoon [4], a Fiat–Shamir signature without rejection sampling whose response is linear [5]–[9] in the secret.

Threshold Raccoon [5] was the first such scheme to reach 1024 signers. Each signer i commits to $\mathbf{w}_i = \mathbf{A}\mathbf{r}_i + \mathbf{e}'_i$ for the public matrix $\mathbf{A}$, a short nonce $\mathbf{r}_i$ and a short noise $\mathbf{e}'_i$, opens it, and answers $\mathbf{z}_i = c\lambda_i \mathbf{s}_i + \mathbf{r}_i + \mathbf{m}_i$ for the challenge c, its Lagrange coefficient λ_i, its key share $\mathbf{s}_i$ and a mask $\mathbf{m}_i$ that cancels in the sum of all answers. This design and its successors leave the following problems open.

Signer state. The masks of Threshold Raccoon are values $\mathsf{PRF}(\mathsf{seed}_{i,j}, \mathsf{sid})$ of a pseudorandom function on a pairwise seed and a session identifier that must never repeat [5], so a signer remembers every identifier it has answered, in a table that grows without bound and that a rollback of a virtual machine resets. It is not known how much state a lattice threshold signer needs at least.

Channels and the first round. In Threshold Raccoon the first round depends on the signer set, and each signer authenticates its view with a signature or a message authentication code. EKT [6] and Ringtail [7] reach two rounds with a preprocessing round, and Ringtail still needs authenticated channels and a preprocessing round that knows the signer set.

Bandwidth. A signer of Threshold Raccoon sends 40.8 KB, and a signer of EKT or Ringtail 276 or 613 KB, mostly in the preprocessing round. Schemes with a smaller footprint restrict the number of signers [9], [10] or leave the Raccoon verifier [8].

Quantum adversaries. The proofs of [5]–[9] read the hash queries of the adversary or rewind it, so they hold in the classical random oracle model, and we are not aware of a lattice threshold signature proven secure under the strongest notion of [3] against quantum adversaries.

Non-terrestrial Networks. As the technologies are preparing for 6G, the 3GPP formally added NTN security study [11], [12], IRTF officially activated SPACE Research Group[2], and mobile networks now integrate low Earth orbit satellite access as NTN [13], whose round trips and capacities change as the satellites move and as the assignment of users to satellites is recomputed every 15 seconds [14], yet to the best of our knowledge no post-quantum scheme has been publicly experimented and measured on such a link.

1. Composing "**Th**" for Threshold and the Turkish word *imza*, meaning "*signature*", Thimza: Threshold Signature.

2. https://dirk-kutscher.info/irtf/ietf-126

1.1. Our Contributions

In this work, we introduce Thimza, a three round lattice threshold signature that solves these problems and keeps the Raccoon verifier. Our contributions are as follows.

- *The state of a signer.* We prove that a threshold signature whose last message is linear in the key share cannot have stateless partial signing, since $t-1$ corrupt parties recover an honest share by replaying its last round (Theorem 3). If the masks are derived from the view of a signer, one token per open session suffices, which is the least state possible (Section 3).
- *The scheme.* The first round of Thimza is independent of the signer set, its masks are antisymmetric pseudorandom values over the digest of the request of the second round, so channels need no authentication, and signers open only the high order bits of their commitments. A signer sends 16.4 KiB at the 128 bit level and $t = 1024$, and signatures of 12.8 KiB are accepted by the unmodified Raccoon verifier (Section 4).
- *Security against quantum adversaries.* We prove the notion TS-UF-4 of Bellare et al. [3], the strongest short of strong unforgeability, in the quantum random oracle model, tightly when a signer counts as having signed once it opens its commitment, and with a loss of Q_s+1 for Q_s signing queries when it counts once it responds, together with correctness, interchangeability with the Raccoon verifier and privacy of the signer set (Sections 4 and 5). The proof combines online extraction [15], one way to hiding [16], adaptive reprogramming [17] and the hint version of the module learning with errors problem MLWE [18] with a new view partition lemma.
- *Mechanized proofs.* `EasyCrypt` [19] checks the protocol, the notions and their hierarchy, and Theorem 8 as one closed lemma over the whole chain of games, with the quantum random oracle modelled as in `EasyPQC` [20] (Section 5.1).
- *Parameters and evaluation.* Our verification bound keeps the bound of the self target module short integer solution problem MSIS below the modulus at every level, which a simpler bound misses at the 192 and 256 bit levels (Section 6). In one framework, a signer of Thimza at $t = 1024$ signs in 60 ms, which is 2.2, 3.3, 12.1 and 26.6 times faster than Threshold Raccoon, TalonG, EKT and Ringtail (Section 7).
- *Measurement on a Starlink link.* We practically deploy and run Thimza on a `LEOScope` node behind a Starlink terminal, send all rounds of a signer across the satellite link at the three levels, and show that at $t = 1024$ receiving the openings, not sending, dominates the latency (Section 7.1).

On a technical level, Thimza is based on Threshold Raccoon, whose verifier, assumptions and parameter shapes it keeps. It keys the masks with the digest of the request that a signer opened instead of a session identifier, so that a mask is tied to the content of a session rather than to its name, and a signer stores this digest before it opens its commitment, so that the challenge of a request is fixed when its last honest member opens. The first property removes identifiers and authenticated channels, and the second lets the proof extract corrupt openings online and program challenges without rewinding.

1.2. Related Work and Applications

Generic constructions. One can evaluate the signing algorithm in multiparty computation [21] or under threshold fully homomorphic encryption [22], [23], which gives one round at a large cost. Damgård et al. [24] gave two round n out of n and multisignature schemes with trapdoor commitments, and Alkeilani Alkadri, Döttling and Pu [25] avoided the restarts of rejection sampling for small committees. Multisignatures such as MuSig-L [26] and DualMS [27] solve a related problem in which every signer has its own key.

The Raccoon family. Espitau, Katsumata and Takemure [6], [28] obtained two rounds with a preprocessed first round under an algebraic one more variant of MLWE, which Zhu and Tessaro related to standard assumptions and showed to give TS-UF-4 [29]. Ringtail [7] reached two rounds under MLWE with a lattice trapdoor and a first round of 588 KiB, Chairattana-Apirom, Tessaro and Zhu [30] used a sharing with short coefficients, and TalonG [8] used an NTRU trapdoor semi-commitment for two rounds at 26.9 KB per signer, with signatures that are not Raccoon signatures. Katsumata, Reichle and Takemure [31] proved adaptive security for a five round variant, and Tweed [32] reached two rounds with adaptive security. Del Pino et al. [33] added identifiable abort to Threshold Raccoon, and Olingo [34] provides it with a distributed key generation, at 852 KB per signer. Espitau, Niot and Prest [35] built the robust scheme Pelican from the hash and sign signature Plover. Hermine [9] replaces masking by a Vandermonde sharing with short shares for up to 64 signers, del Pino and Niot [10] obtained signatures close to one Dilithium signature for up to 8 signers, and Gur, Katz and Silde [36] used threshold homomorphic encryption.

Standardized signatures. Tang et al. [37] evaluate Dilithium in multiparty computation, Quorus [38] does so for ML-DSA with an honest majority, Celi et al. [39] give threshold ML-DSA for up to 6 parties, THED [40] uses threshold homomorphic encryption, and Trilithium [41] and earlier protocols [42], [43] target two signers. Threshold FALCON [44] and Kettle [45] produce hash and sign signatures, partial lattice trapdoors allow noninteractive threshold preimage sampling, two recent preprints propose threshold ML-DSA with shared nonces [46], [47], and [48], [49] survey the structures.

Comparison. Table 1 compares lattice threshold signatures (upper part) with threshold protocols for standardized signatures (middle part). The schemes differ in where the cost of masking moves as Threshold Raccoon pays in two rounds

and authenticates views, EKT and Ringtail pay with large preprocessed first messages, TalonG leaves the Raccoon verifier, and Hermine and the scheme of del Pino and Niot restrict the number of signers. Among the schemes with the Raccoon verifier and 1024 signers, Thimza sends the fewest bytes, 16.4 KiB against 28.2 to 852 KB, and it is the first to combine them with state bounded by the open sessions and less than 20 KB per signer.

2. Preliminaries

The appendices give the complete proofs of all theorems and the details of our measurements.

Notation. Let λ be the security parameter, $[n] = \{1, \ldots, n\}$, and $x \leftarrow D$ denote sampling from a distribution D. The ring is $\mathcal{R} = \mathbb{Z}[X]/(X^\varphi + 1)$ of degree φ, a power of two, and $\mathcal{R}_q = \mathcal{R}/q\mathcal{R}$ for a prime modulus $q \equiv 1 \bmod 2\varphi$. Norms of ring elements are those of their coefficient vectors, extended to vectors by concatenation. For $x \in \mathbb{Z}_q$ with representative in $[0, q)$, the rounding that drops $\nu \geq 1$ bits is $\lfloor x \rceil_\nu = \lfloor (x + 2^{\nu-1})/2^\nu \rfloor \bmod q_\nu$ with $q_\nu = \lfloor q/2^\nu \rfloor$, applied to every coefficient. Our moduli satisfy $q \equiv 1 \bmod 2^{41}$, so $q = q_\nu 2^\nu + 1$ for $\nu \leq 41$, and $2^\nu \lfloor x \rceil_\nu = x + \epsilon$ in $\mathbb{Z}_q$ with a rounding residue $|\epsilon| \leq 2^{\nu-1} + 1$. The discrete Gaussian D_σ of width σ over $\mathbb{Z}$ has weights proportional to $\exp(-x^2/(2\sigma^2))$ and is applied to every coefficient, and $\eta_\varepsilon(\mathbb{Z}^\varphi)$ is the smoothing parameter of $\mathbb{Z}^\varphi$ with error ε for Gaussian weights $\exp(-\pi x^2/s^2)$ of width s [50]. The challenge set $\mathcal{C} = \{c \in \mathcal{R} : \|c\|_\infty = 1,\ \|c\|_1 = \omega\}$ contains the ternary polynomials with ω nonzero coefficients.

Assumptions. Our proofs are based on the following lattice problems and a pseudorandom function.

Definition 1 (MLWE and HintMLWE)**.** For a modulus q, dimensions k, ℓ and a width σ, the problem $\mathsf{MLWE}_{q,k,\ell,\sigma}$ asks to distinguish $(\mathbf{A}, \mathbf{A}\mathbf{s} + \mathbf{e})$ from $(\mathbf{A}, \mathbf{u})$, for a uniform matrix $\mathbf{A} \in \mathcal{R}_q^{k \times \ell}$, a uniform vector $\mathbf{u} \in \mathcal{R}_q^k$ and a secret and noise $(\mathbf{s}, \mathbf{e}) \leftarrow D_\sigma^\ell \times D_\sigma^k$ [51]. The problem $\mathsf{HintMLWE}_{q,k,\ell,Q,\sigma_1,\sigma_2}$ [18] is the same problem with a secret and noise of width σ_1, in which the distinguisher also receives Q hints $(c_j,\ c_j\mathbf{s} + \mathbf{r}_j,\ c_j\mathbf{e} + \mathbf{e}'_j)$ with challenges c_j uniform in $\mathcal{C}$ and hint noise $(\mathbf{r}_j, \mathbf{e}'_j)$ of width σ_2, computed from the same $(\mathbf{s}, \mathbf{e})$ in both worlds.

Lemma 1 reduces HintMLWE to MLWE [18].

Lemma 1. *Let $0 < \varepsilon \leq \frac{1}{2}$ and define the width σ_0 by $\sigma_0^{-2} = 2(\sigma_1^{-2} + Q\omega^2\sigma_2^{-2})$. If $\sigma_0 \geq \eta_\varepsilon(\mathbb{Z}^\varphi)/\sqrt{\pi}$, then $\mathsf{Adv}^{\mathsf{HintMLWE}_{q,k,\ell,Q,\sigma_1,\sigma_2}} \leq \mathsf{Adv}^{\mathsf{MLWE}_{q,k,\ell,\sigma_0}} + 2(k+\ell)\varepsilon$.*

Proof. Kim et al. [18] prove this for Gaussians with weights $\exp(-\pi x^2/s^2)$, which are our D_σ with $s = \sqrt{2\pi}\sigma$, for widths $s_0^{-2} = 2(s_1^{-2} + B_\mathrm{h} s_2^{-2})$ and the condition $s_0 \geq \sqrt{2}\,\eta_\varepsilon(\mathbb{Z}^\varphi)$, where B_h bounds the sum of the squared largest singular values of the multiplications by the c_j. Multiplication by $c \in \mathcal{C}$ is a sum of ω signed permutations, so its largest singular value is at most ω and $B_\mathrm{h} = Q\omega^2$ is admissible. Multiplying by 2π then gives our equation and our condition. □

Definition 2 (Self target MSIS)**.** For a bound β and a rounding amount ν_t, the problem $\mathsf{SelfTargetMSIS}_{q,k,\ell,\mathcal{C},\beta}$ [52] gives a uniform matrix $\mathbf{A} \in \mathcal{R}_q^{k \times \ell}$ and a uniform vector $\mathbf{b} \in \mathcal{R}_q^k$, sets $\mathbf{t} = \lfloor \mathbf{b} \rceil_{\nu_\mathrm{t}}$, and gives quantum access to a random function $G : \mathcal{R}_q^k \times \{0,1\}^* \to \mathcal{C}$. It asks for a string aux and a vector $(\mathbf{z}, c, \mathbf{u})$ with $\|(\mathbf{z}, c, \mathbf{u})\|_2 \leq \beta$ and $G([\mathbf{A} \mid -2^{\nu_\mathrm{t}}\mathbf{t} \mid \mathbf{I}](\mathbf{z}, c, \mathbf{u}), \mathsf{aux}) = c$, where $\mathbf{I}$ is the identity matrix.

A pseudorandom function PRF is secure if no quantum adversary with classical oracle access distinguishes N independent instances from random functions, and $\mathsf{Adv}^{\mathsf{PRF}}(N)$ denotes the advantage of such an adversary.

Quantum random oracles. Hash functions are random functions to which the adversary has quantum access [53], while honest parties query them classically. We use four facts for a random function $H : X \to \{0,1\}^\kappa$ with outputs of κ bits and an algorithm that makes Q queries to it.

- *Collisions* [20], [54]. The probability of outputting $x \neq x'$ with $H(x) = H(x')$ is at most $27(Q+2)^3/2^\kappa$.
- *Hidden points* [16]. If a function G equals H outside a set S, where G, H, S and the input z may have any joint distribution, then $|\Pr[A^H(z) = 1] - \Pr[A^G(z) = 1]| \leq 2Q\sqrt{P}$, where P is the probability that a query of $A^H(z)$, chosen uniformly and measured, lies in S.
- *Adaptive reprogramming* [17]. Reprogramming H at R points with fresh uniform values, where each point is drawn from a distribution chosen by the algorithm with largest probability at most p, changes its output distribution by at most $R(\sqrt{Qp} + Qp/2)$, even if the algorithm learns the points and side information sampled with them.
- *Online extraction* [15]. There is a simulator with an oracle interface $S.\mathsf{RO}$ and an extraction interface $S.\mathsf{E}$ such that, if an algorithm outputs targets $y_1, \ldots, y_L$ that are extracted as $\hat{x}_i = S.\mathsf{E}(y_i)$ and later inputs x_i, then the joint distribution differs from the real one by at most $8L(Q+L)\sqrt{2/2^\kappa}$, and $\Pr[\exists i : x_i \neq \hat{x}_i \wedge H(x_i) = y_i] \leq 8L(Q+1)\sqrt{2/2^\kappa} + (40e^2(Q+L+1)^3 + 2)/2^\kappa$.

Finally, the regularity lemma [55] implies that for all but a $2q^{-\varphi}$ fraction of $\mathbf{A} \in \mathcal{R}_q^{k \times \ell}$, the value $\mathbf{A}\mathbf{r} + \mathbf{e}'$ for $(\mathbf{r}, \mathbf{e}') \leftarrow D_\sigma^\ell \times D_\sigma^k$ is within statistical distance $2^{-\varphi}$ of uniform over $\mathcal{R}_q^k$, as soon as $\sqrt{2\pi}\sigma \geq 2\sqrt{\varphi}\, q^{k/(k+\ell)+2/(\varphi(k+\ell))}$. Hence every function of $\mathbf{A}\mathbf{r} + \mathbf{e}'$ takes each value with probability at most $2^{-\mu}$ for the min entropy $\mu = \varphi - 1$, whenever each value of the function has at most $q^{k\varphi}2^{-\varphi}$ preimages.

Raccoon. Raccoon [4] applies the Fiat–Shamir paradigm to an identification protocol based on MLWE, in which the prover commits to $\mathbf{A}\mathbf{r} + \mathbf{e}'$ for a short nonce $\mathbf{r}$ and noise $\mathbf{e}'$ and answers a challenge $c \in \mathcal{C}$ with $\mathbf{z} = c\mathbf{s} + \mathbf{r}$, and in which low order bits are dropped and a hint fixes the approximation error. A Raccoon key is $\mathsf{vk} = (\mathbf{A}, \mathbf{t})$ with the rounded key $\mathbf{t} = \lfloor \mathbf{A}\mathbf{s} + \mathbf{e} \rceil_{\nu_\mathrm{t}}$ for a secret and noise $(\mathbf{s}, \mathbf{e}) \leftarrow D_{\sigma_\mathrm{t}}^\ell \times D_{\sigma_\mathrm{t}}^k$

TABLE 1. LATTICE THRESHOLD SIGNATURES AT THE 128 BIT LEVEL. SIZES IN KILOBYTES AS REPORTED BY THE AUTHORS, EXCEPT FOR THIMZA, WHOSE SIZES ARE EXACT AND IN KIB. "ROUNDS" IS OFFLINE PLUS ONLINE INTERACTION, "COMM." IS THE BYTES ONE SIGNER SENDS IN ONE EXECUTION (ONLINE PART IN PARENTHESES), AND "VERIFIER" IS THE STANDARD VERIFIER THAT ACCEPTS THE THRESHOLD SIGNATURES, WITH ✗ FOR NONE. "N/R" MEANS NOT REPORTED IN A COMPARABLE FORM, AND LIP IS THE LATTICE ISOMORPHISM PROBLEM.

Scheme	max t	Rounds	Comm.	$\lvert\mathsf{vk}\rvert$	$\lvert\sigma\rvert$	Verifier	Features	Assumptions
BKP [21] (ACNS'13)	small	many	large	n/r	n/r	GPV	generic MPC	SIS
BGG$^+$ [22] (CRYPTO'18)	any	$0+1$	large	n/r	n/r	any	threshold FHE	LWE
ASY [23] (ICALP'22)	any	$0+1$	large	n/r	n/r	any	threshold FHE	LWE
DOTT [24] (PKC'21)	$t=n$	$0+2$	n/r	n/r	n/r	✗	trapdoor commitment	MLWE, MSIS
ADP [25] (ACNS'24)	7, $t=n$	$0+3$	25	7	12	✗		MLWE, MSIS
Threshold Raccoon [5] (EUROCRYPT'24)	1024	$0+3$	40.8	3.9	12.7	Raccoon		MLWE, MSIS
EKT [6], [29] (CRYPTO'24)	1024	$1+1$	276.1 (14.1)	5.5	10.8	Raccoon		AOM-MLWE, MSIS
KRT [31] (CRYPTO'24)	n/r	$0+5$	n/r	n/r	n/r	Raccoon	adaptive	MLWE, MSIS
Pelican [35] (CRYPTO'24)	16	$0+4$	364	12.8	12.3	Plover	robust, DKG	RLWE, RSIS
CTZ [30] (ASIACRYPT'24)	5	$1+1$	1526	47	258	✗		MSIS
GKS [36] (PQCrypto'24)	5	$0+2$	3072	13	46	✗	threshold FHE	RLWE, RSIS
Ringtail [7] (S&P'25)	1024	$1+1$	612.8 (10.5)	4.5	13.4	Raccoon		MLWE, MSIS
dPN [10] (PKC'25)	8	$0+3$	7.5	2.6	2.7	✗		MLWE, MSIS
Unmasked TRaccoon [33] (CRYPTO'25)	1024	$0+3$	28.2	n/r	12.7	Raccoon	ident. abort	MLWE, MSIS
TalonG [8] (EUROCRYPT'26)	1024	$0+2$	26.9	2.0	17.7	✗		RLWE, RSIS, NTRU
Tweed [32] (EUROCRYPT'26)	n/r	$0+2$	n/r	n/r	n/r	n/r	adaptive	MLWE, MSIS
Hermine [9] (ASIACRYPT'26)	64	$1+1$	73.2 (19.5)	2.8	11.3	Raccoon	ident. abort, proactive	MLWE, MSIS
Olingo [34] (CCS'26)	1024	$0+3$	852	4.4	11.3	Raccoon	DKG, ident. abort	MLWE, MSIS
Tang et al. [37] (TIFS'23)	small	many	n/r	1.3	2.4	Dilithium	MPC	MLWE, MSIS
Quorus [38] (USENIX'26)	any	many	≥ 100 per try	1.3	2.4	ML-DSA	honest majority	MLWE, MSIS
Celi et al. [39] (USENIX'26)	6	few	≤ 1000	1.3	2.4	ML-DSA		MLWE, MSIS
THED [40] (CCS'26)	any	$1+1$	23.6	1.3	2.4	ML-DSA	threshold FHE	MLWE, MSIS, others
Trilithium [41] (ESORICS'26)	2	few	n/r	1.3	2.4	ML-DSA	helper party	MLWE, MSIS
Threshold FALCON [44] (CCS'26)	any	many	n/r	0.9	0.7	Falcon	MPC	NTRU
Kettle [45] (Preprint'26)	any	≈ 12	≥ 2 online	1.0	0.6	HAWK	MPC	LIP
Thimza (Our Work)	1024	$0+3$	**16.4**	3.8	12.8	Raccoon	no channel authentication	MLWE, MSIS

of width σ_{t} and a key rounding amount ν_{t}, where $\mathbf{A}$ is expanded from a seed. A signature on a message msg is $(c, \mathbf{z}, \mathbf{h})$ with the response $\mathbf{z} = c\mathbf{s} + \mathbf{r}$, the challenge $c = H_{\mathrm{c}}(\mathsf{vk}, \mathbf{w}, \mathsf{msg})$ of a hash function H_{c} on a commitment $\mathbf{w}$ rounded by ν_{w} bits, and the hint $\mathbf{h} = \mathbf{w} - \lfloor \mathbf{A}\mathbf{z} - 2^{\nu_{\mathrm{t}}} c\mathbf{t} \rceil_{\nu_{\mathrm{w}}}$ [4], [5]. For a verification bound B, the verifier accepts if

$$c = H_{\mathrm{c}}(\mathsf{vk}, \lfloor \mathbf{A}\mathbf{z} - 2^{\nu_{\mathrm{t}}} c\mathbf{t} \rceil_{\nu_{\mathrm{w}}} + \mathbf{h}, \mathsf{msg}) \wedge \|(\mathbf{z}, 2^{\nu_{\mathrm{w}}}\mathbf{h})\|_2 \leq B. \tag{1}$$

Threshold signatures. We use the following syntax and notions of security.

Definition 3 (Three round threshold signature)**.** A three round threshold signature consists of a key generation $\mathsf{KeyGen}(1^\lambda, t, n)$, which outputs a verification key vk and secret keys $\mathsf{sk}_1, \ldots, \mathsf{sk}_n$, a first round $\mathsf{Sign}_1(\mathsf{sk}_i, \mathsf{msg})$, which outputs a token st_i and a first message, rounds Sign_2 and Sign_3, which update or consume the token and output the second and third messages, a combiner $\mathsf{Combine}$ and a verifier Verify. The input of the second round is a request $V = (\mathsf{msg}, T, (\mathsf{cmt}_j)_{j \in T})$, which consists of a message, a signer set T and one first message cmt_j per member.

Definition 4 (Unforgeability)**.** In the game $\mathsf{TS\text{-}UF}\text{-}x[\mathsf{s}]$ for $x \in \{0, \ldots, 4\}$ and semantics $\mathsf{s} \in \{\mathsf{o}, \mathsf{r}\}$, the adversary chooses a set $\mathcal{CS}$ of corrupt parties with $|\mathcal{CS}| \leq t-1$, receives vk and $(\mathsf{sk}_j)_{j \in \mathcal{CS}}$, has quantum access to the hash functions, and may call, for an honest party $i \in \mathcal{H} = [n] \setminus \mathcal{CS}$, the oracles $\mathcal{O}_1(i, \mathsf{msg})$, $\mathcal{O}_2(\mathsf{st}, V)$ and $\mathcal{O}_3(\mathsf{st}, V, (\bar{\mathbf{w}}_j)_{j \in T})$ with openings $\bar{\mathbf{w}}_j$, which run the three rounds on a token of its choice, where $\mathcal{O}_3$ consumes the token. An honest i *answers* V when its $\mathcal{O}_2$ call on V succeeds ($\mathsf{s} = \mathsf{o}$) or when it later responds with the token that opened V ($\mathsf{s} = \mathsf{r}$). Let $S_1(\mathsf{msg})$ be the honest parties that answered some request for msg, $S_2(V)$ those that answered V, $S_3(V)$ the honest members of T whose own first message in V was output by them, and $S_4(V) = \mathcal{H} \cap T$. The adversary wins if it outputs $(\mathsf{msg}^*, \sigma^*)$ with $\mathsf{Verify}(\mathsf{vk}, \mathsf{msg}^*, \sigma^*) = 1$ and no trivial forgery condition holds, where the conditions are $\mathsf{tf}_0: \ S_1(\mathsf{msg}^*) \neq \emptyset$, $\mathsf{tf}_1: \ |S_1(\mathsf{msg}^*)| \geq t - |\mathcal{CS}|$, and for some V with msg^*, $\mathsf{tf}_2: \ |S_2(V)| \geq t - |\mathcal{CS}|$, $\mathsf{tf}_3: \ \mathsf{tf}_2(V) \wedge S_2(V) = S_3(V)$ and $\mathsf{tf}_4: \ \mathsf{tf}_2(V) \wedge S_2(V) = S_4(V)$.

The notions are those of Bellare et al. [3], whose requests become the inputs of our second round. The opening semantics o suits a scheme that binds the request before it opens, because a signer that has opened has committed to everything that determines the challenge.

Figure 1 gives the game. After the adversary $\mathcal{A}$ chooses $\mathcal{CS}$, the challenger generates the keys, starts with empty answer logs L_{o} and L_{r}, and runs $\mathcal{A}$ with the signing oracles and quantum access $|H\rangle$ to the hash functions. The oracle $\mathcal{O}_1$ runs the first round for an honest i and returns a fresh handle u of the token with the first message, $\mathcal{O}_2$ runs the second round on the token u and a request V and, if it succeeds, records V in the token, adds (V, i) to L_{o} and returns the opening $\bar{\mathbf{w}}$, and $\mathcal{O}_3$ runs the third round and, if it succeeds, adds the recorded request to L_{r}, returns the response $\mathbf{z}$ and erases the token. A forgery wins if it verifies and tf_x fails for the log L_{s}.

Game $\mathsf{TS\text{-}UF}\text{-}x[\mathsf{s}]$: $\mathcal{CS} \leftarrow \mathcal{A}$ with $|\mathcal{CS}| \le t-1$; $(\mathsf{vk}, (\mathsf{sk}_i)_i) \leftarrow \mathsf{KeyGen}$; $L_\mathsf{o}, L_\mathsf{r} := \emptyset$; $(\mathsf{msg}^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}_1,\mathcal{O}_2,\mathcal{O}_3,|H\rangle}(\mathsf{vk}, (\mathsf{sk}_j)_{j\in\mathcal{CS}})$; win if $\mathsf{Verify}(\mathsf{vk}, \mathsf{msg}^*, \sigma^*)$ and not tf_x for L_s.

$\mathcal{O}_1(i, \mathsf{msg})$, $i \in \mathcal{H}$: $(\mathsf{st}, \mathsf{cmt}) := \mathsf{Sign}_1(\mathsf{sk}_i, \mathsf{msg})$; store st under a fresh handle u; return (u, cmt).

$\mathcal{O}_2(u, V)$: if $\mathsf{Sign}_2(\mathsf{st}_u, V)$ succeeds with $\bar{\mathbf{w}}$: record V in st_u, add (V, i) to L_o, return $\bar{\mathbf{w}}$.

$\mathcal{O}_3(u, T, (\mathsf{cmt}_j, \bar{\mathbf{w}}_j)_j)$: if Sign_3 succeeds with $\mathbf{z}$: add (V_u, i) to L_r for the request V_u recorded in st_u, return $\mathbf{z}$; st_u is erased.

Figure 1. The unforgeability game. $|H\rangle$ is quantum access to the hash functions; the answer logs L_o and L_r define S_1, S_2 in the two semantics.

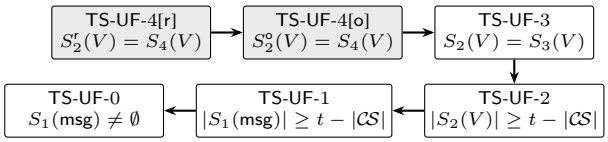


Figure 2. The notions of Definition 4 for Thimza. Each box gives the trivial forgery condition, which for TS-UF-3 and TS-UF-4 is added to the condition $|S_2(V)| \ge t - |\mathcal{CS}|$ of TS-UF-2. An arrow means that security under its source implies security under its target (Lemma 2), and Theorems 8 and 9 prove the two shaded notions.

Trust assumptions and threat model. We trust only a dealer that runs KeyGen and privately delivers the keys, and each honest signer to keep its tokens, its only state, secret and never roll them back (Theorem 3). The adversary statically corrupts up to $t-1$ parties, plays the coordinator and the combiner, and controls the network, whose channels are neither private nor authenticated, so it chooses the signer set, may show its members different requests, and may replay, reorder, delay or drop any message; liveness is not a goal. It interleaves concurrent sessions arbitrarily, may abandon a token in any phase, sees all messages of honest signers, even of incomplete sessions, and may forge on any message, even that of an open session.

Figure 2 orders the notions. Each box names a notion with its trivial forgery condition, where S_2^o and S_2^r denote S_2 in the two semantics, and an arrow means that security under its source implies security under its target, so TS-UF-4 in the response semantics is the strongest notion. Theorems 8 and 9 prove the two shaded notions. Lemma 2 shows that the notions form this chain.

Lemma 2. *For every adversary against Thimza,*

$$\mathsf{Adv}^{\mathsf{TS\text{-}UF\text{-}0}} \le \cdots \le \mathsf{Adv}^{\mathsf{TS\text{-}UF\text{-}4[o]}} \le \mathsf{Adv}^{\mathsf{TS\text{-}UF\text{-}4[r]}}, \quad (2)$$

where the first notions are taken in either semantics.

Proof. An honest signer opens a request only if the entry of the signer in the request is its own first message, so $S_2(V) \subseteq S_3(V) \subseteq S_4(V)$, and $S_2(V) \subseteq S_1(\mathsf{msg})$ for the message msg of V. Hence tf_4 implies tf_3, since $S_2 = S_4$ forces S_3 to equal both, tf_3 implies tf_2 by definition, $\mathsf{tf}_2(V)$ implies tf_1 for the message of V because S_1 contains $S_2(V)$, and tf_1 implies tf_0 because $t - |\mathcal{CS}| \ge 1$. A forgery that avoids a weaker condition avoids every stronger one, so the winning events grow with x. A response uses the token that opened the request, so $S_2^\mathsf{r}(V) \subseteq S_2^\mathsf{o}(V) \subseteq S_4(V)$. If tf_4 holds for V in semantics r, then $S_2^\mathsf{r}(V) = S_4(V)$ forces $S_2^\mathsf{o}(V) = S_4(V)$, and tf_4 holds in semantics o. Hence a winning forgery in semantics o also wins in semantics r. □

The two semantics. They differ only on requests that every honest member of T opened but one never answered in the third round, for instance because it went offline. The adversary then holds every opening and every other response, but not the last response, which carries the last share and nonce. The opening semantics counts a forgery on such a message as trivial, and Theorem 9 shows that such forgeries are hard as well. Both games let the adversary combine many concurrent sessions, so they cover attacks such as those on Schnorr multisignatures [56]. Strong unforgeability in the sense of [3] asks that a request determines at most one valid signature, which for Raccoon signatures would follow from MSIS with a bound of about $2B$, and this bound exceeds q for our parameters as for those of Raccoon, so TS-UF-4 in semantics r is the strongest notion that we consider.

Definition 5 (Privacy of the signer set)**.** A threshold signature hides its signer set if, for all keys, messages and signer sets T_0, T_1 with $|T_0| = |T_1| = t$, the output of Combine in honest executions with T_0 and with T_1 is identically distributed.

3. The State of a Threshold Signer

We first characterize the state that a signer must keep. The class of the next definition contains every Fiat–Shamir lattice threshold signature that we know of, and FROST.

Definition 6 (Linear responses)**.** A threshold signature has *linear responses* if the last message of signer i is $\mathbf{z}_i = c\,\gamma_i(\mathsf{vw}_i)\mathbf{s}_i + f_i(\rho_i, \mathsf{vw}_i) + \mathbf{m}_i(\mathsf{vw}_i)$, where vw_i is the transcript received by i, the challenge c and the unit $\gamma_i(\mathsf{vw}_i) \in \mathcal{R}_q$ are public functions of it, ρ_i is the randomness of the first round, the function f_i does not depend on the share $\mathbf{s}_i$, and the mask $\mathbf{m}_i$ is computed from sk_i and vw_i, with $\sum_{i\in T} \mathbf{m}_i(\mathsf{vw}) = 0$ when all members of T receive the same vw. Partial signing is *stateless* if the last round is a deterministic function of $(\mathsf{sk}_i, \rho_i, \mathsf{vw}_i)$ that may be called on the same ρ_i more than once.

In Theorem 3, we show that stateless partial signing is impossible.

Theorem 3. *Let a threshold signature with linear responses, in which $f_i(\rho_i, \mathsf{vw}_i)$ does not depend on vw_i and the challenge is a random oracle with values in $\mathcal{C}$, have stateless partial signing. An adversary that corrupts $t-1$ parties and makes one query of the first round and $m+1$ queries of the last round to one honest signer recovers the signing key, except with probability at most $\varphi 2^{-m}$.*

Proof sketch. The adversary corrupts $T \setminus \{h\}$ for an honest h and sends h views $\mathsf{vw}_0, \ldots, \mathsf{vw}_m$ that differ only in one corrupt first message, so the challenges c_a are independent and uniform while $\gamma = \gamma_h(\mathsf{vw}_a)$ and, by statelessness, the randomness ρ_h stay the same. It computes the mask of h

from the corrupt masks and obtains $\mathbf{d}_a = (c_a - c_0)\gamma\mathbf{s}_h$ for $a \in [m]$. In each of the φ slots of the number theoretic transform, $c_a - c_0$ vanishes with probability at most $1/2$ by a sign flip argument, so $\gamma\mathbf{s}_h$ is recovered except with probability $\varphi 2^{-m}$, and interpolation with the corrupt shares gives $\mathbf{s}$. Appendix A gives the full proof. □

Two round schemes [6], [7] combine several nonces with an aggregation vector that depends on the view, so f_i depends on vw_i. Corollary 4 extends the attack to aggregated nonces.

Corollary 4. *Theorem 3 also holds if* $f_i(\rho_i, \mathsf{vw}_i) = \mathbf{r}_{i,1} + \sum_{k=2}^{K} b_k(\mathsf{vw}_i)\mathbf{r}_{i,k}$ *for nonces* $\mathbf{r}_{i,k}$ *in* ρ_i *and values* $b_k(\mathsf{vw}_i) \in \mathcal{R}_q$ *of a random oracle that is independent of the challenge, with* $m + K$ *queries of the last round and failure probability at most* $\varphi(2^{1-m-K} + q^{-m})$.

Proof. Let $M = m + K$. As before, the adversary obtains $\mathbf{d}_a = c_a\gamma\mathbf{s}_h + \mathbf{r}_{h,1} + \sum_{k\geq 2} b_{a,k}\mathbf{r}_{h,k}$ for $a \leq M$, with independent uniform aggregation values $b_{a,k} = b_k(\mathsf{vw}_a)$. In slot i, given by the root ζ_i of $X^\varphi + 1$, this is a linear system whose coefficient matrix has the columns $\mathbf{c} = (c_a(\zeta_i))_a$, the all ones vector $\mathbf{1}$ and $K - 1$ uniform columns, and $\gamma\mathbf{s}_h$ is determined in slot i when $\mathbf{c}$ is outside the span of the other columns. By the sign flip argument, $\mathbf{c} \in \mathbb{Z}_q\mathbf{1}$ with probability at most 2^{1-M}. Otherwise each of the q vectors $\mathbf{c} - \alpha\mathbf{1}$ for $\alpha \in \mathbb{Z}_q$ is nonzero and lies in the span of $K - 1$ uniform columns with probability at most q^{K-1-M}, so $\mathbf{c}$ lies in the span of all other columns with probability at most q^{-m}. A union bound over the slots concludes. □

A signer therefore has to remember, for each open session, that its randomness was used, while a signer of Threshold Raccoon stores every identifier that it has ever answered. The next definition captures the least that a signer can store.

Definition 7 (Bounded state)**.** A signer has *bounded state* if its store consists only of tokens created by the first round of open sessions, each consumed by at most one execution of the last round and erased immediately afterwards.

Bounded state suffices once the masks are derived from the view, as in $\mathbf{m}_i(\mathsf{vw}_i) = \sum_{j\in T\setminus\{i\}} \varepsilon_{ij}\mathsf{PRF}(\mathsf{sd}_{\{i,j\}}, \mathsf{vw}_i)$ with pairwise seeds $\mathsf{sd}_{\{i,j\}}$ and signs $\varepsilon_{ij} = -\varepsilon_{ji}$. The masks of signers that saw the same view cancel, a replay of the same view returns the same answer, and a replay of a different view is refused because the token is gone. Our security theorems hold in a model in which the only state of an honest signer is its tokens, so by Theorem 3 Thimza keeps the least state possible for a scheme with linear responses. Constructions from threshold homomorphic encryption [22], [23] are outside this class, because their partial signature is a deterministic function of the key and the message.

4. The Thimza Scheme

In the following, we give an informal and intuitive description of Thimza, whose formal description is in Figure 3. A trusted dealer samples a Raccoon key, shares the secret $\mathbf{s}$ among the n parties with Shamir sharing [1], and gives every pair of parties a common seed. A signature of Thimza is a Raccoon signature under vk, and the core of the protocol is the way in which t parties compute one.

As a first attempt, each member i of the signer set T announces $\mathbf{w}_i = \mathbf{A}\mathbf{r}_i + \mathbf{e}'_i$, all members compute the challenge c from $\sum_{j\in T}\mathbf{w}_j$, and each member answers $\mathbf{z}_i = c\lambda_{T,i}\mathbf{s}_i + \mathbf{r}_i$ with its Lagrange coefficient $\lambda_{T,i}$ for T, so that $\sum_{i\in T}\mathbf{z}_i = c\mathbf{s} + \sum_{i\in T}\mathbf{r}_i$ is a Raccoon response. This protocol has two problems. First, a corrupt member may choose its commitment after it has seen the honest ones, which enables concurrent attacks [56], so Threshold Raccoon first commits to $\mathbf{w}_i$ with a hash and opens it in a second round. Second, a proof cannot simulate a single answer, because the share is multiplied by $\lambda_{T,i}$, which is a large element of $\mathcal{R}_q$, so Threshold Raccoon adds a mask that cancels in the sum and is keyed by a session identifier. Thimza keeps these three rounds and changes three of their elements.

Request bound before opening. The first message is a hash of the truncated commitment and does not depend on T, so the first round can be run for any later signer set. The second round receives the request $V = (\mathsf{msg}, T, (\mathsf{cmt}_j)_{j\in T})$, stores its digest vw and only then opens, and the third round checks every opening against its first message and the request against the stored digest. Corrupt openings are therefore fixed before the honest ones are revealed, and every honest response is tied to the request that it opened.

Masks derived from the request. The digest is the input of the masks, which removes session identifiers and authenticated channels, since signers that received different requests compute masks that do not cancel and whose sum is uniform (Lemma 10). One seed per unordered pair and the signs $\varepsilon_{ij} = -\varepsilon_{ji}$ make the masks of a request sum to zero with $t - 1$ evaluations of PRF per signer and $n - 1$ seeds per key, where Threshold Raccoon uses a row and a column mask and $2n$ seeds. A response whose mask partners are all corrupt is not hidden, so $t - 1$ pairwise values per signer are the least that this technique uses.

Truncated commitments. A signer opens only $\bar{\mathbf{w}}_i = \lfloor\mathbf{w}_i\rceil_{\nu_c}$, its commitment without the ν_c low order bits. The sum $\sum_j 2^{\nu_c}\bar{\mathbf{w}}_j$ differs from $\sum_j \mathbf{w}_j$ by at most $t(2^{\nu_c-1} + 1)$ per coefficient, which the hint absorbs. With $\nu_c = \lfloor\log_2 \sigma_w\rfloor - 1$ for the nonce width σ_w, the second message shrinks from 15 680 to 4 160 bytes at the 128 bit level, while the coordinate variance of the hint grows by 2.1% at every t, since the ratio of the truncation noise to the nonce noise is $2^{\nu_c}/(\sigma_w\sqrt{12})$. Truncation changes neither what is bound nor what is hidden, because H_0 hashes the truncated value, the challenge is computed from truncated values only, and a truncated commitment still takes each value with probability at most $2^{-\mu}$, since each value has at most $(2^{\nu_c} + 1)^{k\varphi} \leq q^{k\varphi}2^{-\varphi}$ preimages.

Thimza **algorithms.** In Figure 3, KeyGen samples a Raccoon key $\mathsf{vk} = (\mathsf{seed}, \mathbf{t})$, shares $\mathbf{s}$ with a uniform polynomial of degree $t - 1$ evaluated at the public points α_i, and gives

$\mathsf{KeyGen}(1^\lambda, t, n)$

1) $\mathsf{seed} \leftarrow \{0,1\}^{2\lambda}$, $\mathbf{A} := \mathsf{Expand}(\mathsf{seed}) \in \mathcal{R}_q^{k\times\ell}$.
2) $(\mathbf{s}, \mathbf{e}) \leftarrow D_{\sigma_\mathrm{t}}^\ell \times D_{\sigma_\mathrm{t}}^k$, $\mathbf{t} := \lfloor \mathbf{A}\mathbf{s} + \mathbf{e} \rceil_{\nu_\mathrm{t}}$, $\mathsf{vk} := (\mathsf{seed}, \mathbf{t})$.
3) $\mathbf{p}_1, \ldots, \mathbf{p}_{t-1} \leftarrow \mathcal{R}_q^\ell$, $\mathbf{s}_i := \mathbf{s} + \sum_{u=1}^{t-1} \alpha_i^u \mathbf{p}_u$ for $i \in [n]$.
4) $\mathsf{sd}_{\{i,j\}} \leftarrow \{0,1\}^{2\lambda}$ for each pair $\{i,j\} \subseteq [n]$.
5) $\mathsf{sk}_i := (\mathbf{s}_i, (\mathsf{sd}_{\{i,j\}})_{j\neq i})$.

$\mathsf{Sign}_1(\mathsf{sk}_i, \mathsf{msg})$

1) $(\mathbf{r}_i, \mathbf{e}_i') \leftarrow D_{\sigma_\mathrm{w}}^\ell \times D_{\sigma_\mathrm{w}}^k$, $\bar{\mathbf{w}}_i := \lfloor \mathbf{A}\mathbf{r}_i + \mathbf{e}_i' \rceil_{\nu_\mathrm{c}}$.
2) $\mathsf{cmt}_i := H_0(\mathsf{vk}, \mathsf{msg}, i, \bar{\mathbf{w}}_i)$.
3) $\mathsf{st}_i := (\mathsf{msg}, \mathbf{r}_i, \bar{\mathbf{w}}_i, \mathsf{cmt}_i, \text{phase } 1)$; send cmt_i.

$\mathsf{Sign}_2(\mathsf{st}_i, T, (\mathsf{cmt}_j)_{j\in T})$

1) Abort unless phase is 1, $|T| = t$, $T \subseteq [n]$, $i \in T$ and the entry of i is cmt_i.
2) $\mathsf{st}_i.\mathsf{vw} := H_1(\mathsf{vk}, \mathsf{msg}, T, (\mathsf{cmt}_j)_{j\in T})$,$\mathsf{st}_i.T := T$, phase:= 2.
3) Send $\bar{\mathbf{w}}_i$.

$\mathsf{Sign}_3(\mathsf{st}_i, T, (\mathsf{cmt}_j, \bar{\mathbf{w}}_j)_{j\in T})$

1) Abort unless phase is 2, $T = \mathsf{st}_i.T$, $H_1(\mathsf{vk}, \mathsf{msg}, T, (\mathsf{cmt}_j)_j) = \mathsf{st}_i.\mathsf{vw}$, $\mathsf{cmt}_j = H_0(\mathsf{vk}, \mathsf{msg}, j, \bar{\mathbf{w}}_j)$ for all $j \in T$, and the entry of i is $\bar{\mathbf{w}}_i$.
2) $\mathbf{w} := \lfloor \sum_{j\in T} 2^{\nu_\mathrm{c}} \bar{\mathbf{w}}_j \rceil_{\nu_\mathrm{w}}$, $c := H_\mathrm{c}(\mathsf{vk}, \mathbf{w}, \mathsf{msg})$.
3) $\mathbf{m}_i := \sum_{j\in T\setminus\{i\}} \varepsilon_{ij} \mathsf{PRF}(\mathsf{sd}_{\{i,j\}}, \mathsf{st}_i.\mathsf{vw})$, with $\varepsilon_{ij} = 1$ if $i < j$ and -1 otherwise.
4) $\mathbf{z}_i := c\lambda_{T,i}\mathbf{s}_i + \mathbf{r}_i + \mathbf{m}_i$; erase st_i; send $\mathbf{z}_i$.

$\mathsf{Combine}(\mathsf{vk}, \mathsf{msg}, T, (\bar{\mathbf{w}}_j, \mathbf{z}_j)_{j\in T})$

1) $\mathbf{w} := \lfloor \sum_{j\in T} 2^{\nu_\mathrm{c}} \bar{\mathbf{w}}_j \rceil_{\nu_\mathrm{w}}$, $c := H_\mathrm{c}(\mathsf{vk}, \mathbf{w}, \mathsf{msg})$, $\mathbf{z} := \sum_{j\in T} \mathbf{z}_j$.
2) $\mathbf{h} := \mathbf{w} - \lfloor \mathbf{A}\mathbf{z} - 2^{\nu_\mathrm{t}} c\mathbf{t} \rceil_{\nu_\mathrm{w}}$, centred modulo q_{ν_w}.
3) Output $\sigma = (c, \mathbf{z}, \mathbf{h})$.

$\mathsf{Verify}(\mathsf{vk}, \mathsf{msg}, \sigma)$

1) Accept if and only if (1) holds, Raccoon verifier with bound B.

Figure 3. The Thimza threshold signature. Round one does not depend on the signer set, round two binds the request, and round three checks the openings, responds and erases the token. Hashes are SHAKE256 with domain separation [57]; H_0, H_1 output $n_H = 5\lambda$ bits, H_c maps to $\mathcal{C}$ and PRF to $\mathcal{R}_q^\ell$. $\lambda_{T,i}$ is the Lagrange coefficient of i for T at 0, and α_i are public points with units as differences.

Round	Signer i sends	Signer i checks before sending
1	cmt_i (80 B)	nothing; T is not yet known
2	$\bar{\mathbf{w}}_i$ (4 160 B)	$\lvert T\rvert = t$, $i \in T$, own cmt_i; stores vw
3	$\mathbf{z}_i$ (12 544 B)	digest = vw, every opening, own $\bar{\mathbf{w}}_i$

Figure 4. One execution at the 128 bit level. Between rounds one and two, any party chooses T among the signers that answered round one and broadcasts the request $V = (\mathsf{msg}, T, (\mathsf{cmt}_j)_{j\in T})$; the combiner adds the $\mathbf{z}_i$ and computes the hint.

party i its share $\mathbf{s}_i$ and its $n-1$ pair seeds. Sign_1 truncates the commitment of a fresh nonce and noise of width σ_w to $\bar{\mathbf{w}}_i$, keeps them in a token of phase 1 and sends the hash cmt_i. Sign_2 checks the phase, $|T| = t$, $i \in T$ and that V carries cmt_i, stores T and the digest vw, and opens $\bar{\mathbf{w}}_i$. Sign_3 checks the phase, T and the digest against the token, every opening against its first message and its own entry against $\bar{\mathbf{w}}_i$, computes $\mathbf{w}$ and c from the truncated openings and the mask from the $t-1$ pair seeds, responds and erases the token. Combine adds the responses and computes the hint centred modulo q_{ν_w}, and Verify is the Raccoon verifier (1).

Figure 4 summarizes one execution at the 128 bit level. Each row gives a round, the message that signer i sends with its size in bytes, and the checks that it performs before sending. Between rounds one and two, a coordinator chooses T among the signers that answered round one, so it can ask more than t parties and keep the first t that answer, and after round three the combiner adds the responses and computes the hint. The third message, a masked and hence uniform element of $\mathcal{R}_q^\ell$, is the largest and is sent uncompressed in $\ell\varphi\lceil\log_2 q\rceil$ bits.

Tokens, aborts and retries. A token holds the message, the nonce, the truncated commitment and the first message, and after round two the digest and the signer set. A check that fails in round two leaves the token in phase 1, so the coordinator may retry, and a check that fails in round three leaves it in phase 2, bound to its digest, so a later third round can only succeed with the same openings and returns the same response. A session in which some member never opens is abandoned by discarding its tokens, which is safe because a nonce whose commitment was opened is never used in a response, and the coordinator can replace a silent member by any party that holds a token of phase 1 for the same message.

An example. Let $T = \{1,2,3\}$ and write $p_{ij} = \mathsf{PRF}(\mathsf{sd}_{\{i,j\}}, \mathsf{vw})$ for the pair values. The masks $\mathbf{m}_1 = p_{12} + p_{13}$, $\mathbf{m}_2 = -p_{12} + p_{23}$ and $\mathbf{m}_3 = -p_{13} - p_{23}$ sum to zero, so $\mathbf{z}_1 + \mathbf{z}_2 + \mathbf{z}_3 = c\mathbf{s} + \mathbf{r}_1 + \mathbf{r}_2 + \mathbf{r}_3$. If the network shows signer 3 a request with another digest vw′, then $\mathbf{z}_1 + \mathbf{z}_2$ contains $p_{13} + p_{23}$ at vw, which no response under vw′ cancels, and $\mathbf{z}_3$ contains $-p_{13} - p_{23}$ at vw′, so all these sums are uniform. With identifiers instead of digests, all three signers would use the same mask values on two different requests.

4.1. Correctness, Interchangeability and Privacy

We now choose the verification bound B so that honest signatures verify except with probability $2^{-\lambda}$, and we show that signatures of Thimza are Raccoon signatures that hide the signer set. Theorem 5 establishes the correctness of Thimza.

Theorem 5. *For every honest execution with $|T| = t$ at most the largest supported threshold T_max, the combiner outputs $(c, \mathbf{z}, \mathbf{h})$ with $c = H_\mathrm{c}(\mathsf{vk}, \lfloor \mathbf{A}\mathbf{z} - 2^{\nu_\mathrm{t}} c\mathbf{t} \rceil_{\nu_\mathrm{w}} + \mathbf{h}, \mathsf{msg})$, $\mathbf{z} = c\mathbf{s} + \sum_{j\in T} \mathbf{r}_j$ and $2^{\nu_\mathrm{w}}\mathbf{h} = \sum_{j\in T} \mathbf{e}_j' + \delta + c(\mathbf{e} + \rho) + \epsilon$, where $\rho = 2^{\nu_\mathrm{t}}\mathbf{t} - \mathbf{A}\mathbf{s} - \mathbf{e}$ and δ and ϵ are rounding residues with $\|\delta\|_\infty \le t(2^{\nu_\mathrm{c}-1} + 1)$ and $\|\epsilon\|_\infty \le 2^{\nu_\mathrm{w}} + 2$. Hence* Verify *accepts exactly when $\|(\mathbf{z}, 2^{\nu_\mathrm{w}}\mathbf{h})\|_2 \le B$. If the rounding residues are modelled as independent uniform variables, as in [4], [5], every coordinate of $(\mathbf{z}, 2^{\nu_\mathrm{w}}\mathbf{h})$ has variance at most $v = T_\mathrm{max}\sigma_\mathrm{w}^2 + \omega\sigma_\mathrm{t}^2 + \omega 4^{\nu_\mathrm{t}}/12 + T_\mathrm{max}4^{\nu_\mathrm{c}}/12 + 4^{\nu_\mathrm{w}}/6$, and $B = \sqrt{v(d + 2\sqrt{d\tau} + 2\tau)}$ with $d = (\ell + k)\varphi$ and $\tau = \lambda \ln 2$ bounds the norm except with probability $2^{-\lambda}$.*

Proof sketch. The masks cancel and Lagrange reconstruction gives $\mathbf{z} = c\mathbf{s} + \sum_j \mathbf{r}_j$. With the sum of the scaled

openings $\mathbf{W} = \sum_j 2^{\nu_c}\bar{\mathbf{w}}_j = \sum_j(\mathbf{A}\mathbf{r}_j + \mathbf{e}'_j) + \delta$ and $2^{\nu_t}\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e} + \rho$, we get $\mathbf{A}\mathbf{z} - 2^{\nu_t}c\mathbf{t} = \mathbf{W} - \mathbf{u}$ for $\mathbf{u} = \sum_j \mathbf{e}'_j + \delta + c(\mathbf{e}+\rho)$. As $\mathbf{h} = \lfloor\mathbf{W}\rceil_{\nu_w} - \lfloor\mathbf{W}-\mathbf{u}\rceil_{\nu_w}$, the verifier recomputes $\lfloor\mathbf{W}\rceil_{\nu_w} = \mathbf{w}$ and c, and two rounding residues give $2^{\nu_w}\mathbf{h} = \mathbf{u} + \epsilon$. In the model, $c\rho$ has ω terms of variance $4^{\nu_t}/12$, δ has t terms of variance $4^{\nu_c}/12$ and ϵ two terms of variance $4^{\nu_w}/12$, which gives v, and the quantile bound of Laurent and Massart [58] for Gaussian coordinates of variance at most v gives the tail $e^{-\tau} = 2^{-\lambda}$. □

Section 6 confirms the model with the exact distributions of the protocol. Theorem 6 addresses interchangeability with the Raccoon verifier.

Theorem 6. Verify *is the Raccoon verifier with bound B, which does not depend on t or T. With $t = n = 1$, the signing protocol computes Raccoon signing with the commitment $2^{\nu_c}\lfloor\mathbf{A}\mathbf{r} + \mathbf{e}'\rceil_{\nu_c}$, and for every t an honest signature is accepted by the Raccoon verifier under the condition of Theorem 5.*

Proof sketch. The verifier Verify is (1) with the parameters of the set. With $t = n = 1$ the sharing is the identity, $\lambda = 1$, the mask is an empty sum, and the rounds compute $c = H_c(\mathsf{vk}, \lfloor 2^{\nu_c}\bar{\mathbf{w}}\rceil_{\nu_w}, \mathsf{msg})$, $\mathbf{z} = c\mathbf{s} + \mathbf{r}$ and the hint of Raccoon, which is the lemma `single_signer` of our development. The last claim is Theorem 5. □

Interchangeability is weaker than functional equivalence, since a signature of Thimza has the nonce $\sum_{j\in T}\mathbf{r}_j$ of width $\sigma_w\sqrt{t}$ and a hint that carries the truncation noise. A relying party only runs the verifier, so interchangeability is what a drop in replacement needs. Theorem 7 shows that signatures hide the signer set.

Theorem 7. *Thimza hides its signer set in the sense of Definition 5.*

Proof sketch. By Theorem 5, the output is $(c, c\mathbf{s} + \sum_{j\in T}\mathbf{r}_j, \mathbf{h})$ with c and $\mathbf{h}$ computed from $\sum_{j\in T} 2^{\nu_c}\lfloor\mathbf{A}\mathbf{r}_j + \mathbf{e}'_j\rceil_{\nu_c}$. Both sums are symmetric functions of t independent pairs $(\mathbf{r}_j, \mathbf{e}'_j)$ drawn from the same distribution, so their joint distribution depends on t only. □

Theorems 5 to 7 are checked in `EasyCrypt`, the last for a full honest execution that passes every check of the three rounds.

5. Security Analysis

Here, we show that Thimza is TS-UF-4 unforgeable [3] in the quantum random oracle model. Let Q_s bound the calls to the signing oracles, and let q_0, q_1, q_c bound the quantum queries of the adversary to the hash functions H_0, H_1, H_c. For the largest threshold $T_{\max}$, we write $Q_0 = q_0 + Q_s(T_{\max}+1)$, $Q_1 = q_1 + 2Q_s$ and $Q_c = q_c + Q_s + 1$ for the numbers of queries including those of honest parties, $L = Q_s T_{\max}$ for the number of extractions, and $\mu = \varphi - 1$. The statistical loss is

$$\epsilon = \tfrac{27((Q_0+2)^3+(Q_1+2)^3)+Q_s^2}{2^{n_H}} + 16L(Q_0+L+1)\sqrt{2^{1-n_H}} + \tfrac{40e^2(Q_0+L+1)^3+2}{2^{n_H}} + 4Q_0\sqrt{Q_s 2^{-\mu}} + Q_s\big(\sqrt{Q_c 2^{-\mu}} + \tfrac{Q_c}{2^{\mu+1}}\big) + \tfrac{5Q_s^2+Q_0Q_s}{2^\mu} + \tfrac{2}{q^\varphi},$$

and the bound of the SelfTargetMSIS instance is $\beta = B + (2^{\nu_w-1}+2)\sqrt{k\varphi} + \sqrt{\omega}$. Theorem 8 states unforgeability in the opening semantics, and Theorem 9 goes through the response semantics.

TABLE 2. BASE TWO LOGARITHM OF THE STATISTICAL TERMS OF THEOREM 8 FOR $q_0 = q_1 = q_c = 2^{64}$ AND THE BUDGETS Q_s OF TABLE 3.

Term	Level I	Level III	Level V
Collisions of H_0, H_1	-425.2	-748.1	-1073.7
Online extraction	-174.5	-336.5	-500.4
Hidden points (G_5)	-153.5	-155.0	-157.8
Adaptive reprogramming (G_6)	-163.5	-164.5	-166.5
Distinct points, early checks	-381.0	-382.9	-386.8
Total ϵ	-153.5	-155.0	-157.8

Theorem 8. *In the quantum random oracle model, for every quantum adversary $\mathcal{A}$ against* TS-UF-4[o] *of Thimza there are adversaries $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3$ with about the running time of $\mathcal{A}$ such that*

$$\mathsf{Adv}^{\mathsf{TS\text{-}UF\text{-}4[o]}}_{\mathcal{A}} \le \mathsf{Adv}^{\mathsf{PRF}}_{\mathcal{B}_1}\big(\tbinom{n}{2}\big) + \mathsf{Adv}^{\mathsf{HintMLWE}_{q,k,\ell,Q_s,\sigma_t,\sigma_w}}_{\mathcal{B}_2} + \mathsf{Adv}^{\mathsf{SelfTargetMSIS}_{q,k,\ell,\mathcal{C},\beta}}_{\mathcal{B}_3} + \epsilon.$$

Theorem 9. *Under the same conditions, there are $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_4$ such that*

$$\mathsf{Adv}^{\mathsf{TS\text{-}UF\text{-}4[r]}}_{\mathcal{A}} \le \mathsf{Adv}^{\mathsf{PRF}}_{\mathcal{B}_1}\big(\tbinom{n}{2}\big) + (Q_s+1)\mathsf{Adv}^{\mathsf{HintMLWE}}_{\mathcal{B}_2} + (Q_s+1)\mathsf{Adv}^{\mathsf{SelfTargetMSIS}}_{\mathcal{B}_4} + \epsilon + Q_s\big(\sqrt{Q_c 2^{-\mu}} + \tfrac{Q_c}{2^{\mu+1}}\big).$$

By (2) both theorems bound every notion from TS-UF-0 to TS-UF-4. With $t = n = 1$ and no corruption, Theorem 8 has no PRF and extraction terms and shows that Raccoon with the parameters of Table 3 is existentially unforgeable in the quantum random oracle model, which is the guarantee that a relying party with the Raccoon verifier expects. As for Dilithium [52], [59], we estimate the hardness of SelfTargetMSIS by that of the corresponding MSIS problem.

Table 2 evaluates the terms of ϵ, grouped by the games of the proof below. The hidden point term $4Q_0\sqrt{Q_s 2^{-\mu}}$ dominates, and $\epsilon \le 2^{-153}$ at every level, so the computational terms determine the security.

We now sketch the proof of Theorem 8, and Appendix B gives the full proof. To understand our strategy, recall how Threshold Raccoon is proven secure in the classical random oracle model [5]. Its reduction reads the hash queries of the adversary to learn the corrupt commitments, programs the challenge of each session, simulates the sum of the responses with a hint of HintMLWE, relies on the masks to make individual responses uniform, and turns a forgery into

a solution of SelfTargetMSIS. Against a quantum adversary this strategy meets two problems. The reduction cannot record queries, so it does not see the corrupt commitments, and it must decide which nonce comes from a hint before the commitment is opened, while a signer cannot know when it opens whether it completes the request. To overcome the first problem, we extract the corrupt openings online [15], which works because every honest signer binds the whole request before it opens. To overcome the second, we let honest first messages be uniform values and program H_0 only when a token opens, which one way to hiding [16] makes invisible. When the last honest member of a request V opens, the aggregate commitment $\mathbf{w}(V)$ and the challenge point $x_V = (\mathsf{vk}, \mathbf{w}(V), \mathsf{msg})$ depend on one fresh nonce and on values already fixed, so x_V has μ bits of min entropy and can be reprogrammed to the challenge of a fresh hint [17].

Definition 8 (Complete request)**.** A request $V = (\mathsf{msg}, T, (\mathsf{cmt}_j)_{j\in T})$ is *complete* once every member of $\mathcal{H} \cap T$ has opened V in a successful call of $\mathcal{O}_2$, and its *last opener* is the token whose call completed it.

In the opening semantics a complete request satisfies $S_2(V) = S_4(V)$ and $|S_2(V)| = t - |T \cap \mathcal{CS}| \geq t - |\mathcal{CS}|$, so every forgery on its message is trivial. In the simulation of the responses, a request plays the role of a session identifier of Threshold Raccoon. Lemma 10 simulates the responses under one request.

Lemma 10. *Fix a request V with digest vw and honest members $\mathcal{H}_T = \mathcal{H} \cap T$, and assume that the values $\mathsf{PRF}(\mathsf{sd}_{\{i,j\}}, \mathsf{vw})$ for honest pairs are uniform and used by no other request. Let the members of $\mathcal{H}_T$ that respond under V do so in the order $i_1, i_2, \ldots$. Every response by a member that is not the last of $\mathcal{H}_T$ to respond is uniform over $\mathcal{R}_q^\ell$ and independent of all other values, and if all of $\mathcal{H}_T$ respond, the last response equals $c\sum_{h\in\mathcal{H}_T} \lambda_{T,h}\mathbf{s}_h + \sum_{h\in\mathcal{H}_T} \mathbf{r}_h + \mathbf{m}_{\mathcal{CS}}(V)$ minus the earlier ones, where $\mathbf{m}_{\mathcal{CS}}(V) = -\sum_{j\in T\cap\mathcal{CS}} \mathbf{m}_j(\mathsf{vw})$ is computed from corrupt seeds.*

Proof. Number the members of $\mathcal{H}_T$ as $0, \ldots, m-1$ in the order in which they respond, placing members that never respond at the end, and write $u_{ab} = \mathsf{PRF}(\mathsf{sd}_{\{a,b\}}, \mathsf{vw})$ for $a < b$, which are independent and uniform by assumption. The mask of member a is $\mathbf{m}_a = \sum_{b\neq a} \pi_{ab} + \mathbf{m}_a^{\mathcal{CS}}$, where $\pi_{ab} = u_{ab}$ if $a < b$ and $\pi_{ab} = -u_{ba}$ if $b < a$, and $\mathbf{m}_a^{\mathcal{CS}}$ collects the terms keyed by seeds shared with corrupt members, which the adversary knows. Since $\pi_{ab} + \pi_{ba} = 0$ and the masks of all members of T sum to zero,

$$\sum_{a<m} \mathbf{m}_a = \sum_{a<m} \mathbf{m}_a^{\mathcal{CS}} = \mathbf{m}_{\mathcal{CS}}(V). \tag{3}$$

The response $\mathbf{z}_a = c\lambda_{T,a}\mathbf{s}_a + \mathbf{r}_a + \mathbf{m}_a$ of a member $a < m-1$ contains $u_{a,m-1}$ with coefficient 1, and $u_{a,m-1}$ occurs in no other value except $\mathbf{z}_{m-1}$, which is not yet issued. Conditioned on all values except $(u_{a,m-1})_{a<m-1}$, the map $(u_{a,m-1})_{a<m-1} \mapsto (\mathbf{z}_a)_{a<m-1}$ is a translation of $\mathcal{R}_q^{\ell(m-1)}$, so $(\mathbf{z}_a)_{a<m-1}$ is uniform and independent of everything else. If the last member also responds, (3) gives $\mathbf{z}_{m-1} = \sum_{a<m}(c\lambda_{T,a}\mathbf{s}_a + \mathbf{r}_a) + \mathbf{m}_{\mathcal{CS}}(V) - \sum_{a<m-1} \mathbf{z}_a$. This is the equivalence `class_sim` of our `EasyCrypt` development, and a class that never completes is `class_prefix`. □

Generally, the proof uses the following games, where $\Pr[G_i]$ is the probability that $\mathcal{A}$ wins G_i, and each game changes the previous one as stated.

- G_0 is TS-UF-4[o] for Thimza, and G_1 replaces $\mathsf{PRF}(\mathsf{sd}_{\{i,j\}}, \cdot)$ by random functions for honest pairs, which costs $\mathsf{Adv}^{\mathsf{PRF}}_{\mathcal{B}_1}(\binom{n}{2})$.
- G_2 aborts if two distinct opened requests have equal digests, or if $\mathcal{O}_3$ accepts, at an entry that is the first message of an honest token, an opening other than the truncated commitment of that token, which costs $27((Q_0+2)^3 + (Q_1+2)^3)2^{-n_H}$ by the collision bound.
- G_3 answers H_0 on corrupt indices with the extractable simulator of [15] and extracts the corrupt first messages of a request when it becomes complete, and G_4 aborts if $\mathcal{O}_3$ accepts a corrupt opening other than the extracted one, which together cost $16L(Q_0+L+1)\sqrt{2^{1-n_H}} + (40e^2(Q_0+L+1)^3 + 2)2^{-n_H}$.
- G_5 gives every token a uniform first message, samples its nonce only when it opens and then programs H_0 to map the truncated commitment to the first message, and aborts if two tokens get the same first message or if $\mathcal{O}_3$ accepts an opening at the entry of a token that has not opened, which costs $4Q_0\sqrt{Q_s 2^{-\mu}} + 2Q_s^2 2^{-\mu} + Q_0 Q_s 2^{-\mu} + Q_s^2 2^{-n_H}$ by the hidden point bound.
- In G_6 the last opener u of a complete request V takes as nonce the noise of a fresh hint $(c_u, c_u\mathbf{s} + \mathbf{r}_u, c_u\mathbf{e} + \mathbf{e}'_u)$, and $H_{\mathrm{c}}(x_V)$ is reprogrammed to c_u, which costs $Q_s(\sqrt{Q_{\mathrm{c}}2^{-\mu}} + Q_{\mathrm{c}}2^{-\mu-1}) + Q_s^2 2^{-\mu}$ by adaptive reprogramming.
- G_7 answers $\mathcal{O}_3$ without the honest shares and masks, since by Lemma 10 every response is uniform except the last one of a request, which the hint gives as in (4), and G_8 samples the corrupt shares independently of $\mathbf{s}$, which changes nothing.
- G_9 replaces $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$ by a uniform vector, which costs $\mathsf{Adv}^{\mathsf{HintMLWE}_{q,k,\ell,Q_s,\sigma_{\mathrm{t}},\sigma_{\mathrm{w}}}}_{\mathcal{B}_2}$, since $(\mathbf{s}, \mathbf{e})$ is used only through $\mathbf{b}$ and at most Q_s hints.

In G_9 the reduction $\mathcal{B}_3$ samples the key and the hints itself, since $\mathbf{b}$ is independent of them. A forgery at a programmed point belongs to a complete request and is trivial, and any other forgery $(c^*, \mathbf{z}^*, \mathbf{h}^*)$ with recomputed commitment $\mathbf{w}^*$ satisfies $[\mathbf{A} \mid -2^{\nu_{\mathrm{t}}}\mathbf{t} \mid \mathbf{I}](\mathbf{z}^*, c^*, \mathbf{u}^*) = 2^{\nu_{\mathrm{w}}}\mathbf{w}^*$ for $\mathbf{u}^* = 2^{\nu_{\mathrm{w}}}\mathbf{w}^* - (\mathbf{A}\mathbf{z}^* - 2^{\nu_{\mathrm{t}}}c^*\mathbf{t})$, whose norm is at most $\|2^{\nu_{\mathrm{w}}}\mathbf{h}^*\|_2 + (2^{\nu_{\mathrm{w}}-1}+2)\sqrt{k\varphi}$ because $q \equiv 1 \bmod 2^{\nu_{\mathrm{w}}}$. It therefore solves SelfTargetMSIS with bound β, and the sum of the costs with the failure probability $2q^{-\varphi}$ of the regularity lemma gives Theorem 8.

Figure 5 shows the oracles of G_9 as $\mathcal{B}_3$ implements them.

$\mathcal{O}_1(i, \mathsf{msg})$: create token u and return a uniform cmt_u.

$\mathcal{O}_2(u, V)$: run the checks of Sign_2. If u is the last honest member of T to open V: extract the corrupt openings of V, open $\bar{\mathbf{w}}_u = \lfloor \mathbf{A}\mathbf{y}_u + \mathbf{y}'_u - c_u\mathbf{b} \rceil_{\nu_c}$ from hint u, compute $x_V = (\mathsf{vk}, \mathbf{w}(V), \mathsf{msg})$ and program $H_c(x_V) := c_u$. Otherwise sample a nonce and open its truncated commitment $\bar{\mathbf{w}}_u$. In both cases program $H_0^{\mathcal{H}}$ to map $(\mathsf{vk}, \mathsf{msg}, i, \bar{\mathbf{w}}_u)$ to cmt_u.

$\mathcal{O}_3(u, T, (\mathsf{cmt}_j, \bar{\mathbf{w}}_j)_j)$: run the checks of Sign_3. If some honest member of T has not responded under V yet: return a uniform $\mathbf{z} \in \mathcal{R}_q^\ell$. Otherwise return the last response (4), computed from $\mathbf{y}_u$, the honest nonces, the corrupt shares and seeds, and the earlier responses.

H_c: $H_c(\mathsf{vk}', \mathbf{w}, \mathsf{msg}') = G(2^{\nu_w}\mathbf{w}, (\mathsf{vk}', \mathsf{msg}'))$ except at programmed points.

Figure 5. The signing oracles of G_9, as implemented by the reduction $\mathcal{B}_3$ to SelfTargetMSIS, where $H_0^{\mathcal{H}}$ is H_0 on honest indices. The key $(\mathbf{s}, \mathbf{e})$ enters only through the hints $(c_u, \mathbf{y}_u, \mathbf{y}'_u)$, which the reduction samples itself because $\mathbf{b}$ is uniform and independent of them.

Proof sketch of Theorem 9. A forgery may now reuse the programmed point of a complete request that some honest member never answered. The proof guesses which of the at most Q_s programmed points the forgery uses, or none, and leaves the guessed request unprogrammed. Its responses except the last are uniform by Lemma 10, and the last is requested only if all honest members answer, which makes a forgery at its point trivial. The guess is made while $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$, where the opening of the last opener does not depend on its challenge, so the factor $Q_s + 1$ applies to the HintMLWE and SelfTargetMSIS terms, and a correct guess costs one reprogramming. Appendix C gives the full proof.

Tightness and the quantum model. Hints are consumed only by openings, so there are at most Q_s of them. The proofs of [5]–[9] read the hash queries of the adversary or rewind it. Our proof does neither, so it holds in the quantum random oracle model, and every hop costs an additive term, whereas the forking lemma of [7]–[9] loses about a square root.

5.1. Mechanized Proofs

Our machine-checked proof development is available at https://anonymous.4open.science/r/thimza/easycrypt-proofs/README.md.

Our `EasyCrypt` [19] development has 774 proved statements, checked with `EasyCrypt r2025.03`, Z3 and Alt-Ergo. It contains the protocol of Figure 3 with its checks and erasure, Theorems 3 and 5 to 7, the notions with (2), Lemma 10, and Theorem 8 as one closed lemma, `thimza_main` in Figure 6. Its proof is a chain of games from TS-UF-4[o] with Thimza that follows Appendix B, where H_0 is drawn as two independent functions on honest and corrupt indices (`dfh_split`): random pair values (`hop_prf`), responses without honest pair values (`g1_g1s`), lazy first messages (`g1t_lazy`), extraction (`ga_ext`), reprogramming (`ge_rep`), responses from hints (`gp_gh`), and the Shamir and HintMLWE steps with the final reduction (`gh_bound`), together with the bounds of all abort events. The loss `eps_main` is an explicit sum of eight terms of the kinds of ϵ, namely collisions, hidden points, extraction, reprogramming and distinct points, with the query numbers of the mechanized model. The lemma holds for every adversary that terminates and respects the query bounds `A_bnd`, for an injective encoding of the inputs of H_1, for openings and aggregate commitments that take each value with probability at most `pw` and `pw2`, which regularity bounds by $2^{-\mu}$, for evaluation points that are units with unit differences, and under the rounding bound of the final step (`rounding_fact`). Theorem 9 is not mechanized.

```
declare axiom A_bnd (H <: HOr {-A, -Cnt})
                    (O <: SOr {-A, -Cnt}) :
  hoare [A(CntH(H), CntO(O)).forge :
  Cnt.c0 = 0 /\ Cnt.c1 = 0 /\ Cnt.cc = 0 /\ Cnt.cs = 0
  ==> Cnt.c0 <= qh0 /\ Cnt.c1 <= qh1
      /\ Cnt.cc <= qhc /\ Cnt.cs <= qsig].

lemma thimza_main &m : rounding_fact =>
  Pr[G0(A).main() @ &m : res]
  <= `|Pr[MPRFG(B1(A)).real() @ &m : res]
       - Pr[MPRFG(B1(A)).ideal() @ &m : res]|
   + `|Pr[HMLWE(B2(A)).main(true) @ &m : res]
       - Pr[HMLWE(B2(A)).main(false) @ &m : res]|
   + Pr[STM(B3(A)).main() @ &m : res]
   + eps_main.
```

Figure 6. Theorem 8 in `EasyCrypt`. `G0` is TS-UF-4[o] with Thimza, `MPRFG`, `HMLWE` and `STM` are the games of PRF, HintMLWE and SelfTargetMSIS with the reductions `B1`, `B2` and `B3`, and `A_bnd` bounds the queries of `A`, counted by `CntH` and `CntO`.

As in `EasyPQC` [20], an oracle is a function drawn from the uniform distribution `dfun` over functions when a game starts and reached only through a wrapper that counts queries, and each fact assumes a query bound given as a Hoare judgement, such as `A_bnd` for the counters `Cnt` of the adversary. The facts specific to quantum adversaries [15]–[17], [54], namely collisions, hidden points, adaptive reprogramming and online extraction, are 10 axioms of two abstract theories, each also true for classical adversaries, which keeps the theories consistent, and the only other axioms state that the abstract distributions are lossless or uniform and that messages and digest inputs form finite types.

6. Concrete Security Parameters

We now derive concrete parameters for Thimza at the 128, 192 and 256 bit post-quantum security levels, called levels I, III and V, for up to $T_{\max} = 1024$ signers.

Table 3 gives the parameters of Thimza at the 128, 192 and 256 bit levels, called levels I, III and V, for up to $T_{\max} = 1024$ signers. The ring, the module dimensions, the widths, the rounding amounts and the challenge weights are those of Threshold Raccoon [5], and ν_c is new. At every level $\beta < q$, and the 124 bits of MLWE at level I are comparable to the 123 bits of Dilithium at its second level.

Widths and rounding. The width σ_w must satisfy the hint condition of Lemma 1 for the budget Q_s, the modulus must hold the aggregate response, of width $\sigma_w\sqrt{T_{\max}} = 2^{42}$, inside $q/2$ after the two roundings, the key rounding must keep $\omega 2^{\nu_t - 1}$ below the aggregate noise, and ν_c must keep

TABLE 3. PARAMETERS OF THIMZA WITH $\varphi = 512$, $q = 2^{49} - 2^{41} + 1$, $\sigma_t = 2^{20}$, $\sigma_w = 2^{37}$, $\nu_c = 36$ AND $T_{\max} = 1024$. HARDNESS IS CLASSICAL CORE SVP [60], SIZES ARE IN BYTES AT $t = 1024$.

	Level I	Level III	Level V
(ℓ, k)	$(4, 5)$	$(6, 7)$	$(7, 8)$
ω	19	31	44
$\log_2 \lvert\mathcal{C}\rvert$	132.8	196.0	256.5
(ν_t, ν_w)	$(37, 40)$	$(36, 40)$	$(35, 41)$
Q_s	2^{60}	2^{59}	2^{57}
σ_0 of Lemma 1	4.76	4.13	5.82
B/q	0.621	0.748	0.823
β/q	0.670	0.806	0.949
MLWE hardness	124	210	264
SelfTargetMSIS hardness	156	244	287
$\lvert\mathsf{vk}\rvert$	3 872	5 856	7 200
$\lvert\sigma\rvert$	13 152	19 568	22 272
Round 1 message	80	120	160
Round 2 message	4 160	5 824	6 656
Round 3 message	12 544	18 816	21 952
Total per signer	16 784	24 760	28 768

TABLE 4. HONEST SIGNATURES SAMPLED WITH THE EXACT DISTRIBUTIONS OF THE PROTOCOL, WITH THE LARGEST $\|(\mathbf{z}, 2^{\nu_w}\mathbf{h})\|_2/B$ OVER THE TRIALS AND THE RATIO OF THE EMPIRICAL COORDINATE VARIANCE OF $2^{\nu_w}\mathbf{h}$ TO THE MODEL OF THEOREM 5.

Level	t	Trials	Largest norm$/B$	Variance ratio
I	16	200	0.132	0.998
I	128	100	0.321	1.000
I	1024	40	0.884	0.999
III	1024	20	0.883	1.005
V	1024	20	0.864	0.999

the truncation noise small against σ_w. At level I the variance v of Theorem 5 is $2^{84}(1+0.0015+0.0208+0.0104)$, where the corrections come from the key rounding, the truncation and the two roundings of the hint, so the truncation costs 1% of B.

The modulus. The prime $q = 2^{49} - 2^{41} + 1$ supports the number theoretic transform for $\varphi = 512$ and satisfies $q \equiv 1 \bmod 2^{41}$. The largest prime below 2^{49} with $q \equiv 1 \bmod 1024$ has $q \bmod 2^{40}$ close to 2^{40}, so rounding to 40 bits wraps around and the residues reach $1.5 \cdot 2^{\nu_w}$.

The bound. We set B by Theorem 5. A simpler bound with an additive slack of $2^{\nu_w+1}\sqrt{k\varphi}$ for the hint gives $\beta \geq q$ at levels III and V, for which SelfTargetMSIS is easy, while our bound leaves $\beta \leq 0.95q$. In Table 4, the largest norm at $t = 1024$ is at most $0.88B$, every signature verifies, and the variance is within 0.5% of v, which confirms the model; for small t the model slightly overestimates the variance, because the two roundings of the hint are then correlated.

The budget. Lemma 1 with $\varepsilon = 2^{-\lambda}$ requires $\sigma_t^{-2} + Q_s\omega^2\sigma_w^{-2} \leq \pi/(2\eta_\varepsilon(\mathbb{Z}^\varphi)^2)$, with $\eta_\varepsilon(\mathbb{Z}^\varphi) \leq \sqrt{\ln(2\varphi(1+1/\varepsilon))/\pi}$, which allows Q_s up to $2^{61.2}$, $2^{59.3}$ and $2^{57.9}$, and we use the largest powers of two. We estimate the MLWE instance of width σ_0 and the SelfTargetMSIS instance with matrix $[\mathbf{A} \mid -2^{\nu_t}\mathbf{t} \mid \mathbf{I}]$ and bound β by the primal attack and the MSIS attack of the lattice estimator [61], whose implementation gives 118.6, 182.2 and 255.2 bits for the three Kyber sets and 120.9 bits for Dilithium at its second level, against 118, 183, 256 and 123 published [59], [62]. Taking $Q_s = 2^{58}$ raises the MLWE hardness at level I to 131 bits, and the key itself, of width $\sigma_t = 2^{20}$, is far harder to recover.

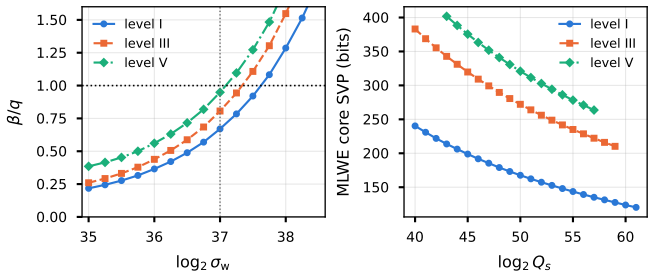


Figure 7. Left: the bound β of the SelfTargetMSIS instance relative to q against the nonce width, with $\nu_c = \lfloor \log_2 \sigma_w \rfloor - 1$ and the other parameters of Table 3. Right: the hardness of the MLWE instance given by Lemma 1 against the budget Q_s, for $\sigma_w = 2^{37}$.

Figure 7 shows the two sides of this choice. The bound β grows linearly with σ_w and reaches q at $\sigma_w = 2^{37.63}$, $2^{37.34}$ and $2^{37.09}$ at levels I, III and V, while the largest budget grows with σ_w^2, so 2^{37} is the largest power of two that is admissible at every level. The budget then trades against the hardness of MLWE, by about 4.4 bits per doubling of Q_s at level I, and a key rotated after 2^{50} signing calls gives 168, 272 and 321 bits.

7. Implementation and Practical Evaluation

All of our practical findings are available at the repository of https://anonymous.4open.science/r/thimza. We conducted our benchmark on an Apple MacBook Air (2022) with the M2 chip, A15 Bionic (64-bit ARM-based), an 8-core CPU featuring a base clock speed of 3.49 GHz, and 16GB of memory, Tahoe OS 26.2 build 25C56, where we measure one signer as the best of three runs for t from 2 to 1024, and Hermine up to its limit of 64. We implemented Thimza, Threshold Raccoon, which tables and figures abbreviate as TRaccoon, EKT, Ringtail, TalonG and Hermine in one Python framework with NumPy that shares the ring arithmetic, the samplers, the hashes and the serializer, so differences come from the schemes. The arithmetic uses a vectorized negacyclic transform, and a uniform element of $\mathcal{R}_q$ takes $\varphi\lceil\log_2 q\rceil$ bits while a Gaussian one of width σ takes $\varphi\lceil\log_2 \sigma + 2.6\rceil$ bits. In our code, every scheme produces signatures that its verifier accepts for five pairs (t, n), and an independent verifier with arithmetic accepts those of Thimza.

On the logarithmic axes of Figure 8, our Thimza has the lowest signing time and communication at every t, and its communication does not depend on t.

Implementation. Uniform elements, including the matrix and the masks, are sampled exactly by rejection on SHAKE256 output, secret randomness comes from a SHAKE256 generator seeded by the operating system, and every hash is domain separated and length prefixed, with

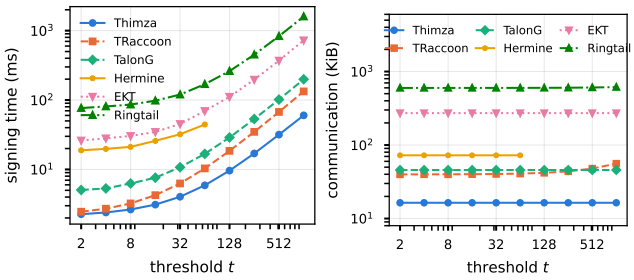


Figure 8. Signing time per signer (left) and bytes sent per signer (right) at the 128 bit level, in one framework and with one serializer.

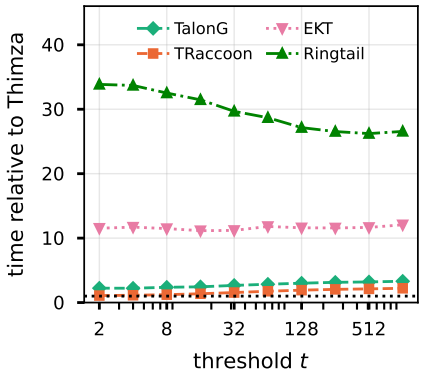


Figure 9. Signing time per signer relative to Thimza at the 128 bit level.

an injective encoding of the truncated commitment in the inputs of H_0. A signer refuses a token in the wrong phase, a request without its own first message, a digest that differs from the stored one, and any opening that does not match its first message. Our tests check each refusal, the cancellation of masks, reconstruction, and that a split view yields an invalid signature.

Figure 9 shows the signing time of TalonG, Threshold Raccoon, EKT and Ringtail divided by that of Thimza against t, where the dotted line marks equal time. Every curve stays above this line, and the advantage over Threshold Raccoon grows from 1.1 at $t = 2$ to 2.2 at $t = 1024$, because Thimza evaluates half as many masks and none in its first round (Table 7).

In Table 5, the first round of Thimza takes 1.2 ms at every t, the second only hashes, and the $t - 1$ masks take three quarters of the 58.6 ms of the third round at $t = 1024$.

In Table 6, where TalonG has no set at level III, Thimza sends 3.0 to 3.4 times less than Threshold Raccoon, whose 55.8 KiB at level I include its authentication tags, with the same Raccoon signatures. TalonG sends 45.8 KiB in our serializer against 26.9 KB in its own encoding, and the two round schemes send most of their bytes before the message is known. Key generation for $n = 1024$ takes 0.63 s against 1.96 s for Threshold Raccoon, because a signer stores $n - 1$ seeds instead of $2n$.

Higher security levels and latency. Table 8 shows that the ratio to Threshold Raccoon stays 2.2 at $t = 1024$ at every level, because both schemes grow with ℓ in the same way. The combiner takes 2.0 to 6.7 ms and the Raccoon verifier

TABLE 5. TIME IN MILLISECONDS OF ONE SIGNER PER ROUND, AT $t = 64$ AND $t = 1024$.

	$t = 64$				$t = 1024$			
Scheme	R1	R2	R3	Total	R1	R2	R3	Total
Thimza	1.2	0.0	4.6	**5.9**	1.2	0.4	58.6	**60.2**
TRaccoon	3.9	0.1	6.4	10.4	43.1	1.2	88.9	133.2
TalonG	2.6	14.1		16.8	2.6	196.4		199.1
Hermine	8.4	35.9		44.4				
EKT	14.0	55.7		69.6	13.1	713.2		726.3
Ringtail	26.4	142.8		169.2	25.6	1572.7		1598.3

TABLE 6. KIB SENT PER SIGNER AT $t = 1024$ (HERMINE AT ITS MAXIMUM $t = 64$) AND SIGNATURE SIZE IN BYTES, IN OUR FRAMEWORK.

	KiB per signer			Signature (bytes)		
Scheme	I	III	V	I	III	V
Thimza	**16.4**	**24.2**	**28.1**	13 152	19 568	22 272
TRaccoon	55.8	74.2	83.4	13 152	19 568	22 272
TalonG	45.8		92.5	21 536		43 072
Hermine	72.5	150.8	219.5	12 832	19 632	25 152
EKT	271.9	346.9	564.2	14 176	18 800	23 424
Ringtail	614.5	772.1	1213.0	14 208	21 424	30 336

2.0 ms at level I. With round trips of 100 ms and 100 Mbit/s links, the model rounds $\times$ 100 ms + bytes/bandwidth + compute gives 361 ms for Thimza at $t = 1024$, 438 ms for Threshold Raccoon, 403 ms for TalonG, and 827 and 1700 ms for the online phase of EKT and Ringtail.

7.1. Measurement on a Satellite Link

LEOScope [63] is a testbed of measurement computers, called nodes, that organizations place behind Starlink user terminals [64], so that the traffic of a node reaches the Internet through satellites in low Earth [65] orbit. An experimenter submits a container image and a configuration to an orchestrator, which starts the container on the requested node at the scheduled instant, stops it after a fixed duration and returns every file that the container wrote.

We measured Thimza via `LEOScope`, here we provide insights, and Appendix I gives the complete details of our experiment[3].

We ran Thimza's implementation with a measurement driver as input to `LEOScope`, as one job on the node `starlink-london-ns2`, an Intel Core i7-1185G7 behind a terminal in London on 5 November 2026. In a session the node plays signer 1 of $T = \{1, \ldots, t\}$ and sends each round across the satellite link to a public measurement server of Cloudflare [66] in one exchange, which uploads the real encoded message of the round and downloads exactly as many bytes as the broadcast that follows it, that is the request V, the t openings, and a valid signature that the node then verifies. The exchanges use TLS 1.3 [67] with the hybrid key exchange X25519MLKEM768 [68], and the job ran 231 sessions in random order over the post-quantum security

3. Input codes and output results are available at https://anonymous.4open.science/r/thimza/NTN_Experiment-LEOScope/README.md.

TABLE 7. DOMINANT OPERATIONS OF ONE SIGNER: N IS A NUMBER THEORETIC TRANSFORM OF ONE RING ELEMENT, P ONE PSEUDORANDOM ELEMENT OF $\mathcal{R}_q^\ell$, H ONE HASH OF A MESSAGE OF THE PROTOCOL, rep THE NUMBER OF COMMITMENTS OF EKT AND $\bar{d}+1$ THE NUMBER OF COMMITMENT COLUMNS OF RINGTAIL.

Scheme	Round	Dominant operations
Thimza	1	$(k+\ell)\mathsf{N}+\mathsf{H}$
	2	H
	3	$(2\ell+1)\mathsf{N}+(t-1)\mathsf{P}+(t+2)\mathsf{H}$
TRaccoon	1	$(k+\ell)\mathsf{N}+t\mathsf{P}$
	2	t message authentication codes
	3	$(2\ell+1)\mathsf{N}+t\mathsf{P}$
EKT	offline	$k\ell\,\mathsf{rep}\,\mathsf{N}$
	online	$k\,\mathsf{rep}\,\mathsf{N}+2t\mathsf{P}$
Ringtail	offline	$k(\ell+1)(\bar{d}+1)\mathsf{N}$
	online	$k\bar{d}\,\mathsf{N}+2t\mathsf{P}$

TABLE 8. SIGNING TIME IN MILLISECONDS OF ONE SIGNER AT THE THREE LEVELS.

	Thimza			TRaccoon		
Level	$t=16$	128	1024	$t=16$	128	1024
I	3.1	9.6	60.2	4.3	18.5	133.2
III	4.2	13.2	83.9	5.7	25.9	182.0
V	4.7	14.8	95.3	6.5	28.8	207.4

levels I, III and V and $t \in \{2, 16, 128, 1024\}$, with real peers up to $t = 16$ and, above, peers that send the messages of signer 1 under their own indices, as in our benchmarks. The same job checked the correctness of the implementation on the node, repeated our benchmarks of Thimza, and sent single network packets of the size of every message.

Table 9 shows the latency of the 231 sessions. The round trip of this link, 24 ms at the median as measured by the TCP stack of the server, is a quarter of the 100 ms of the model above, so a session at $t = 2$ takes only 0.53 to 0.62 times what the model predicts with the signing time of the node, but a session at $t = 1024$ takes 1.43 to 1.66 times that prediction, because a signer then receives 243 to 259 times as many bytes as it sends and the round that delivers the t openings takes 55 to 59 percent of the session. Only the first message of Thimza fits in one packet, because the path carried 119 of 120 echo requests of at most 1500 bytes and none of 240 larger ones, which must be split into IP fragments that networks often drop [69], so the other messages need a transport that splits them into packets, such as TCP [70], which our sessions use, or QUIC [71]. The 10 sessions that crossed one of the instants at which the constellation reassigns terminals to satellites, every 15 seconds [14], took 1.32 times the median of their configuration.

Lessons. On such a link, a message should be judged against the 1500-byte packet of the path and not only against the bandwidth, and for large signer sets the latency comes from the t openings that a signer downloads rather than from what it sends or computes, so the bytes that a signer receives are the figure that matters for threshold signatures in NTN.

TABLE 9. SIGNING SESSIONS OF ONE SIGNER OVER THE STARLINK LINK AT SECURITY LEVEL L, WITH 30, 30, 12 AND 5 SESSIONS FOR $t = 2$, 16, 128 AND 1024, IN MS UNLESS NOTED. RECV. IS THE KILOBYTES (10^3 BYTES) RECEIVED IN A SESSION, COMP. THE SIGNING TIME IN IT, R1 TO R3 THE NETWORK TIMES OF THE THREE EXCHANGES, SESSION THE TIME FROM ROUND ONE UNTIL THE SIGNER HOLDS THE SIGNATURE AND P95 ITS 95TH PERCENTILE, BOTH WITHOUT THE SERVER'S OWN PROCESSING TIME (MEDIAN 109 TO 176 MS), AND RATE THE GOODPUT OF THE OPENINGS IN MBIT/S. ALL VALUES EXCEPT RECV. AND P95 ARE MEDIANS.

L	t	Recv.	Comp.	R1	R2	R3	Session	p95	Rate
I	2	19.4	11.5	44.9	51.1	53.0	**162.0**	219.9	13.7
	16	79.2	15.0	44.1	59.4	58.2	**176.4**	216.4	40.6
	128	555.0	22.8	46.0	147.6	52.4	**294.3**	557.9	40.2
	1024	4356.6	103.0	80.2	329.5	52.6	**555.7**	1599.8	115.6
III	2	28.2	15.2	43.3	52.3	78.4	**190.9**	219.8	17.4
	16	112.1	19.9	40.7	77.7	77.4	**216.7**	293.0	20.9
	128	779.0	30.4	40.4	156.8	77.8	**308.6**	341.9	52.3
	1024	6107.7	136.2	71.8	360.0	75.7	**648.6**	744.3	147.8
V	2	32.5	17.1	43.6	51.4	59.4	**176.1**	349.9	16.7
	16	128.7	22.5	44.5	87.1	57.3	**213.6**	275.4	21.6
	128	893.2	40.8	41.5	162.6	64.9	**305.8**	377.2	60.0
	1024	7003.3	141.8	91.8	419.0	78.4	**724.1**	1975.1	146.2

8. Discussion

Deployment. The only state of a signer is the set of tokens of its open sessions, and a signer can serve any number of concurrent sessions, since tokens are independent. A restored snapshot can bring back a consumed token, which leaks the share by Theorem 3, so tokens live in memory that is not snapshotted or carry a counter checked against a monotonic source. A snapshot then exposes at most the sessions open when it was taken, whereas a restored registry of identifiers makes every past identifier usable again.

The signing budget. The budget Q_s counts all calls to honest signers, including those of sessions that never complete or fail a check, so a deployment counts calls rather than signatures and deals a new key before Q_s calls, and Figure 7 shows the hardness that a smaller budget buys.

Conclusion and Future Directions

Thimza signers keep one token per open session, the least possible for linear responses, need no authenticated channels and send 16.4 KiB at $t = 1024$, while producing Raccoon signatures that are TS-UF-4 secure against quantum adversaries. The masks derived from view of Thimza carry over to two round designs [6], [7], though combining them with truncation is an open work, and identifiable abort [33], distributed key generation [34], [35] and adaptive corruption [31], [32] are our future steps. Mechanizing Theorem 9 would improve our machine checked proofs of both notions. Also, on satellite links, the t openings that each signer downloads are the next cost to cut.

References

[1] A. Shamir, "How to share a secret," *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.

[2] C. Komlo and I. Goldberg, "FROST: Flexible round-optimized Schnorr threshold signatures," in *Selected Areas in Cryptography (SAC 2020)*, ser. LNCS, vol. 12804. Springer, 2021, pp. 34–65.

[3] M. Bellare, E. Crites, C. Komlo, M. Maller, S. Tessaro, and C. Zhu, "Better than advertised security for non-interactive threshold signatures," in *Advances in Cryptology – CRYPTO 2022, Part IV*, ser. LNCS, vol. 13510. Springer, 2022, pp. 517–550.

[4] R. del Pino, S. Katsumata, T. Prest, and M. Rossi, "Raccoon: A masking-friendly signature proven in the probing model," in *Advances in Cryptology – CRYPTO 2024, Part I*, ser. LNCS, vol. 14920. Springer, 2024, pp. 409–444.

[5] R. del Pino, S. Katsumata, M. Maller, F. Mouhartem, T. Prest, and M.-J. O. Saarinen, "Threshold Raccoon: Practical threshold signatures from standard lattice assumptions," in *Advances in Cryptology – EUROCRYPT 2024, Part II*, ser. LNCS, vol. 14652. Springer, 2024, pp. 219–248.

[6] T. Espitau, S. Katsumata, and K. Takemure, "Two-round threshold signature from algebraic one-more learning with errors," in *Advances in Cryptology – CRYPTO 2024, Part VII*, ser. LNCS, vol. 14926. Springer, 2024, pp. 387–424.

[7] C. Boschini, D. Kaviani, R. W. F. Lai, G. Malavolta, A. Takahashi, and M. Tibouchi, "Ringtail: Practical two-round threshold signatures from learning with errors," in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 149–164.

[8] L. Gao, G. Tang, D. Jia, Y. Liu, B. Liu, X. Lu, K. Wang, and Y. Yin, "TalonG: Bandwidth-efficient two-round threshold signatures from lattices," in *Advances in Cryptology – EUROCRYPT 2026, Part I*, ser. LNCS, vol. 16541. Springer, 2026, pp. 94–123.

[9] G. Borin, S. Celi, R. del Pino, T. Espitau, S. Katsumata, G. Niot, T. Prest, and K. Takemure, "Hermine: An efficient lattice-based FROST-like threshold signature," Advances in Cryptology – ASIACRYPT 2026, to appear. Cryptology ePrint Archive, Paper 2026/419, 2026. [Online]. Available: https://eprint.iacr.org/2026/419

[10] R. del Pino and G. Niot, "Finally! A compact lattice-based threshold signature," in *Public-Key Cryptography – PKC 2025, Part III*, ser. LNCS, vol. 15676. Springer, 2025, pp. 169–199.

[11] WhatTheSpec, "3GPP TSG SA WG3 Meeting #130 Meeting Summary," https://whatthespec.net/summarizemeetings/m/TSGS3_130_Prague, 2026, accessed: 2026-11-07.

[12] 3GPP TSG SA WG3, "Tdoc S3-264594: Contribution to 3GPP TSG SA WG3 Meeting #130," https://www.3gpp.org/ftp/tsg_sa/WG3_Security/TSGS3_130_Prague/Docs/S3-264594.zip, 2026, accessed: 2026-11-07.

[13] O. Kodheli, E. Lagunas, N. Maturo, S. K. Sharma, B. Shankar, J. F. M. Montoya, J. C. M. Duncan, D. Spano, S. Chatzinotas, S. Kisseleff, J. Querol, L. Lei, T. X. Vu, and G. Goussetis, "Satellite communications in the new space era: A survey and future challenges," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 1, pp. 70–109, 2021.

[14] H. B. Tanveer, M. Puchol, R. Singh, A. Bianchi, and R. Nithyanand, "Making sense of constellations: Methodologies for understanding Starlink's scheduling algorithms," in *Companion of the 19th International Conference on Emerging Networking EXperiments and Technologies (CoNEXT)*. ACM, 2023, pp. 37–43.

[15] J. Don, S. Fehr, C. Majenz, and C. Schaffner, "Online-extractability in the quantum random-oracle model," in *Advances in Cryptology – EUROCRYPT 2022, Part III*, ser. LNCS, vol. 13277. Springer, 2022, pp. 677–706.

[16] A. Ambainis, M. Hamburg, and D. Unruh, "Quantum security proofs using semi-classical oracles," in *Advances in Cryptology – CRYPTO 2019, Part II*, ser. LNCS, vol. 11693. Springer, 2019, pp. 269–295.

[17] A. B. Grilo, K. Hövelmanns, A. Hülsing, and C. Majenz, "Tight adaptive reprogramming in the QROM," in *Advances in Cryptology – ASIACRYPT 2021, Part I*, ser. LNCS, vol. 13090. Springer, 2021, pp. 637–667.

[18] D. Kim, D. Lee, J. Seo, and Y. Song, "Toward practical lattice-based proof of knowledge from Hint-MLWE," in *Advances in Cryptology – CRYPTO 2023, Part V*, ser. LNCS, vol. 14085. Springer, 2023, pp. 549–580.

[19] G. Barthe, B. Grégoire, S. Heraud, and S. Zanella Béguelin, "Computer-aided security proofs for the working cryptographer," in *Advances in Cryptology – CRYPTO 2011*, ser. LNCS, vol. 6841. Springer, 2011, pp. 71–90.

[20] M. Barbosa, G. Barthe, X. Fan, B. Grégoire, S.-H. Hung, J. Katz, P.-Y. Strub, X. Wu, and L. Zhou, "EasyPQC: Verifying post-quantum cryptography," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2021, pp. 2564–2586.

[21] R. Bendlin, S. Krehbiel, and C. Peikert, "How to share a lattice trapdoor: Threshold protocols for signatures and (H)IBE," in *Applied Cryptography and Network Security (ACNS 2013)*, ser. LNCS, vol. 7954. Springer, 2013, pp. 218–236.

[22] D. Boneh, R. Gennaro, S. Goldfeder, A. Jain, S. Kim, P. M. R. Rasmussen, and A. Sahai, "Threshold cryptosystems from threshold fully homomorphic encryption," in *Advances in Cryptology – CRYPTO 2018, Part I*, ser. LNCS, vol. 10991. Springer, 2018, pp. 565–596.

[23] S. Agrawal, D. Stehlé, and A. Yadav, "Round-optimal lattice-based threshold signatures, revisited," in *49th International Colloquium on Automata, Languages, and Programming (ICALP 2022)*, ser. LIPIcs, vol. 229. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022, pp. 8:1–8:20.

[24] I. Damgård, C. Orlandi, A. Takahashi, and M. Tibouchi, "Two-round n-out-of-n and multi-signatures and trapdoor commitment from lattices," in *Public-Key Cryptography – PKC 2021, Part I*, ser. LNCS, vol. 12710. Springer, 2021, pp. 99–130.

[25] N. Alkeilani Alkadri, N. Döttling, and S. Pu, "Practical lattice-based distributed signatures for a small number of signers," in *Applied Cryptography and Network Security (ACNS 2024), Part I*, ser. LNCS, vol. 14583. Springer, 2024, pp. 376–402.

[26] C. Boschini, A. Takahashi, and M. Tibouchi, "MuSig-L: Lattice-based multi-signature with single-round online phase," in *Advances in Cryptology – CRYPTO 2022, Part II*, ser. LNCS, vol. 13508. Springer, 2022, pp. 276–305.

[27] Y. Chen, "DualMS: Efficient lattice-based two-round multi-signature with trapdoor-free simulation," in *Advances in Cryptology – CRYPTO 2023, Part V*, ser. LNCS, vol. 14085. Springer, 2023, pp. 716–747.

[28] T. Espitau, S. Katsumata, and K. Takemure, "Two-round threshold signature from algebraic one-more learning with errors," *Journal of Cryptology*, vol. 38, no. 4, p. 31, 2025.

[29] C. Zhu and S. Tessaro, "The algebraic one-more MISIS problem and applications to threshold signatures," in *Advances in Cryptology – CRYPTO 2025, Part I*, ser. LNCS, vol. 16000. Springer, 2025, pp. 548–581.

[30] R. Chairattana-Apirom, S. Tessaro, and C. Zhu, "Partially non-interactive two-round lattice-based threshold signatures," in *Advances in Cryptology – ASIACRYPT 2024, Part IV*, ser. LNCS, vol. 15487. Springer, 2025, pp. 268–302.

[31] S. Katsumata, M. Reichle, and K. Takemure, "Adaptively secure 5 round threshold signatures from MLWE/MSIS and DL with rewinding," in *Advances in Cryptology – CRYPTO 2024, Part VII*, ser. LNCS, vol. 14926. Springer, 2024, pp. 459–491.

[32] K. Jiang, S. Tessaro, H. Wee, and C. Zhu, "Tweed: Adaptively secure lattice-based two-round threshold signatures," in *Advances in Cryptology – EUROCRYPT 2026, Part I*, ser. LNCS, vol. 16541. Springer, 2026, pp. 124–153.

[33] R. del Pino, S. Katsumata, G. Niot, M. Reichle, and K. Takemure, "Unmasking TRaccoon: A lattice-based threshold signature with an efficient identifiable abort protocol," in *Advances in Cryptology – CRYPTO 2025, Part VI*, ser. LNCS, vol. 16005. Springer, 2025, pp. 423–456.

[34] K. D. Gur, P. Hough, J. Katz, C. Sandsbråten, and T. Silde, "Olingo: Threshold lattice signatures with DKG and identifiable abort," ACM Conference on Computer and Communications Security (CCS) 2026, to appear. Cryptology ePrint Archive, Paper 2025/1789, 2026. [Online]. Available: https://eprint.iacr.org/2025/1789

[35] T. Espitau, G. Niot, and T. Prest, "Flood and submerse: Distributed key generation and robust threshold signature from lattices," in *Advances in Cryptology – CRYPTO 2024, Part VII*, ser. LNCS, vol. 14926. Springer, 2024, pp. 425–458.

[36] K. D. Gur, J. Katz, and T. Silde, "Two-round threshold lattice-based signatures from threshold homomorphic encryption," in *Post-*

Quantum Cryptography (PQCrypto 2024), Part II, ser. LNCS, vol. 14772. Springer, 2024, pp. 266–300.

[37] G. Tang, B. Pang, L. Chen, and Z. Zhang, "Efficient lattice-based threshold signatures with functional interchangeability," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 4173–4187, 2023.

[38] A. Bienstock, L. de Castro, D. Escudero, A. Polychroniadou, and A. Takahashi, "Quorus: Efficient, scalable threshold ML-DSA signatures from MPC," in *35th USENIX Security Symposium (USENIX Security 26)*. USENIX Association, 2026, pp. 6405–6424.

[39] S. Celi, R. del Pino, T. Espitau, G. Niot, and T. Prest, "Efficient threshold ML-DSA," in *35th USENIX Security Symposium (USENIX Security 26)*. USENIX Association, 2026, pp. 6305–6324.

[40] J. H. Park, A. Passelègue, and D. Stehlé, "THED: Threshold Dilithium from FHE," ACM Conference on Computer and Communications Security (CCS) 2026, to appear. Cryptology ePrint Archive, Paper 2026/638, 2026. [Online]. Available: https://eprint.iacr.org/2026/638

[41] S.-S. Kravtšenko, P. Laud, and N. Snetkov, "Trilithium: Efficient and universally composable distributed ML-DSA signing," in *Computer Security – ESORICS 2026, Part II*, ser. LNCS, vol. 16969. Springer, 2027, pp. 502–521.

[42] J. Vakarjuk, N. Snetkov, and J. Willemson, "DiLizium: A two-party lattice-based signature scheme," *Entropy*, vol. 23, no. 8, p. 989, 2021.

[43] N. Snetkov, J. Vakarjuk, and P. Laud, "TOPCOAT: Towards practical two-party Crystals-Dilithium," *Discover Computing*, vol. 27, no. 1, p. 18, 2024.

[44] R. Garg, D. Escudero, A. Polychroniadou, A. Takahashi, and X. Wang, "Thresholdizing standardized FALCON signatures," ACM Conference on Computer and Communications Security (CCS) 2026, to appear. Cryptology ePrint Archive, Paper 2026/1300, 2026. [Online]. Available: https://eprint.iacr.org/2026/1300

[45] C. Abou Haidar, D. Escudero, T. Espitau, C. Hoffmann, K. Takemure, M. Tibouchi, and H. D. Vanegas Madrigal, "Kettle: Short post-quantum threshold signatures from the HAWK signature scheme," Cryptology ePrint Archive, Paper 2026/2288, 2026. [Online]. Available: https://eprint.iacr.org/2026/2288

[46] L. Kao, "FIPS 204-compatible threshold ML-DSA via Shamir nonce DKG," arXiv preprint 2601.20917, 2026. [Online]. Available: https://arxiv.org/abs/2601.20917

[47] L. Kao and R. Chang, "TALUS: FIPS-204-exact threshold ML-DSA via boundary clearance," arXiv preprint 2603.22109, 2026. [Online]. Available: https://arxiv.org/abs/2603.22109

[48] K. Sedghighadikolaei and A. A. Yavuz, "A survey of threshold signatures: NIST standards, post-quantum cryptography, exotic techniques, and real-world applications," *ACM Computing Surveys*, vol. 58, no. 6, pp. 1–39, 2026.

[49] F. De Sclavis, M. Nardelli, and M. Pedicini, "Thresholding post-quantum signatures," arXiv preprint 2608.26792, 2026. [Online]. Available: https://arxiv.org/abs/2608.26792

[50] D. Micciancio and O. Regev, "Worst-case to average-case reductions based on Gaussian measures," *SIAM Journal on Computing*, vol. 37, no. 1, pp. 267–302, 2007.

[51] A. Langlois and D. Stehlé, "Worst-case to average-case reductions for module lattices," *Designs, Codes and Cryptography*, vol. 75, no. 3, pp. 565–599, 2015.

[52] E. Kiltz, V. Lyubashevsky, and C. Schaffner, "A concrete treatment of Fiat-Shamir signatures in the quantum random-oracle model," in *Advances in Cryptology – EUROCRYPT 2018, Part III*, ser. LNCS, vol. 10822. Springer, 2018, pp. 552–586.

[53] D. Boneh, Ö. Dagdelen, M. Fischlin, A. Lehmann, C. Schaffner, and M. Zhandry, "Random oracles in a quantum world," in *Advances in Cryptology – ASIACRYPT 2011*, ser. LNCS, vol. 7073. Springer, 2011, pp. 41–69.

[54] M. Zhandry, "How to record quantum queries, and applications to quantum indifferentiability," in *Advances in Cryptology – CRYPTO 2019, Part II*, ser. LNCS, vol. 11693. Springer, 2019, pp. 239–268.

[55] V. Lyubashevsky, C. Peikert, and O. Regev, "A toolkit for Ring-LWE cryptography," in *Advances in Cryptology – EUROCRYPT 2013*, ser. LNCS, vol. 7881. Springer, 2013, pp. 35–54.

[56] F. Benhamouda, T. Lepoint, J. Loss, M. Orrù, and M. Raykova, "On the (in)security of ROS," in *Advances in Cryptology – EUROCRYPT 2021, Part I*, ser. LNCS, vol. 12696. Springer, 2021, pp. 33–53.

[57] National Institute of Standards and Technology, "SHA-3 standard: Permutation-based hash and extendable-output functions," NIST, Federal Information Processing Standards Publication 202, 2015.

[58] B. Laurent and P. Massart, "Adaptive estimation of a quadratic functional by model selection," *The Annals of Statistics*, vol. 28, no. 5, pp. 1302–1338, 2000.

[59] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Dilithium: A lattice-based digital signature scheme," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2018, no. 1, pp. 238–268, 2018.

[60] E. Alkim, L. Ducas, T. Pöppelmann, and P. Schwabe, "Post-quantum key exchange: A new hope," in *25th USENIX Security Symposium*. USENIX Association, 2016, pp. 327–343.

[61] M. R. Albrecht, R. Player, and S. Scott, "On the concrete hardness of Learning with Errors," *Journal of Mathematical Cryptology*, vol. 9, no. 3, pp. 169–203, 2015.

[62] J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Kyber: A CCA-secure module-lattice-based KEM," in *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2018, pp. 353–367.

[63] S. Fadaei, S. Tiwari, A. Taneja, S. Bhushan, M. Kassem, A. Raman, D. Bhattacherjee, L. Qiu, A. Woodward, and N. Sastry, "LEOScope: Building a global testbed for low-Earth orbit satellite networks," *ACM SIGCOMM Computer Communication Review*, vol. 55, no. 2, pp. 13–22, 2025.

[64] F. Michel, M. Trevisan, D. Giordano, and O. Bonaventure, "A first look at Starlink performance," in *Proceedings of the 22nd ACM Internet Measurement Conference (IMC)*. ACM, 2022, pp. 130–136.

[65] N. Mohan, A. E. Ferguson, H. Cech, R. Bose, P. R. Renatin, M. K. Marina, and J. Ott, "A multifaceted look at Starlink performance," in *Proceedings of the ACM Web Conference 2024*. ACM, 2024, pp. 2723–2734.

[66] Cloudflare, "Cloudflare speed test," https://speed.cloudflare.com, 2026, endpoints `/__up` and `/__down`, accessed November 2026.

[67] E. Rescorla, "The transport layer security (TLS) protocol version 1.3," RFC 8446, Internet Engineering Task Force, 2018.

[68] K. Kwiatkowski, P. Kampanakis, B. Westerbaan, and D. Stebila, "Post-quantum hybrid ECDHE-MLKEM key agreement for TLSv1.3," Internet-Draft draft-ietf-tls-ecdhe-mlkem, Internet Engineering Task Force, 2025.

[69] R. Bonica, F. Baker, G. Huston, R. M. Hinden, O. Troan, and F. Gont, "IP fragmentation considered fragile," RFC 8900, Internet Engineering Task Force, 2020.

[70] W. M. Eddy, "Transmission control protocol (TCP)," RFC 9293, Internet Engineering Task Force, 2022.

[71] J. Iyengar and M. Thomson, "QUIC: A UDP-based multiplexed and secure transport," RFC 9000, Internet Engineering Task Force, 2021.

[72] National Institute of Standards and Technology, "Module-lattice-based key-encapsulation mechanism standard," Federal Information Processing Standards Publication 203, 2024.

[73] R. T. Fielding, M. Nottingham, and J. Reschke, "HTTP/1.1," RFC 9112, Internet Engineering Task Force, 2022.

[74] World Wide Web Consortium, "Server timing," W3C Working Draft, https://www.w3.org/TR/server-timing/, accessed November 2026.

Appendix A. Theorem 3 Proof

Slots and signs. Since $q \equiv 1 \bmod 2\varphi$, the polynomial $X^\varphi + 1$ has φ distinct roots $\zeta_1, \ldots, \zeta_\varphi$ in $\mathbb{Z}_q$, and the map $a \mapsto (a(\zeta_i))_i$ is a ring isomorphism $\mathcal{R}_q \cong \mathbb{Z}_q^\varphi$, whose components we call slots. We use one fact about the challenge set. For every i and every $v \in \mathbb{Z}_q$, a uniform $c \in \mathcal{C}$ satisfies $\Pr[c(\zeta_i) = v] \le \frac{1}{2}$. To see this, write $c = \sum_{j \in J} \pm X^j$ for a uniform support J of size ω and independent uniform signs. Fix J, an index $j_0 \in J$ and all signs except the one of X^{j_0}. The two remaining choices give values of $c(\zeta_i)$ that differ

by $2\zeta_i^{j_0}$, which is nonzero because ζ_i is a unit and q is odd, so at most one of them equals v.

The attack. Fix an honest party h and a signer set $T \ni h$ of size t, and let the adversary corrupt $T \setminus \{h\}$, so that it knows the keys of all other members of T. It obtains the first message of h, completes it with first messages of the corrupt members, and builds views $\mathsf{vw}_0, \ldots, \mathsf{vw}_m$ for the last round of h that contain the same signer set and the same first message of h but differ in the first message of one corrupt member. The unit $\gamma = \gamma_h(\mathsf{vw}_a)$ depends on T only and is the same for all a, and the challenges c_a are values of the random oracle at distinct points, hence independent and uniform in $\mathcal{C}$. Because partial signing is stateless, h answers every vw_a with the same randomness ρ_h, and because f_h does not depend on the view, $\mathbf{z}_a = c_a\gamma\mathbf{s}_h + f_h(\rho_h) + \mathbf{m}_h(\mathsf{vw}_a)$. The masks of T cancel and the adversary computes the masks of the other members itself, so it obtains $\mathbf{m}_h(\mathsf{vw}_a) = -\sum_{j\in T\setminus\{h\}} \mathbf{m}_j(\mathsf{vw}_a)$ and

$$\mathbf{d}_a = (\mathbf{z}_a - \mathbf{m}_h(\mathsf{vw}_a)) - (\mathbf{z}_0 - \mathbf{m}_h(\mathsf{vw}_0)) = (c_a - c_0)\gamma\mathbf{s}_h.$$

Recovery of the key. Write $\mathbf{x} = \gamma\mathbf{s}_h$. In slot i, every coordinate satisfies $\mathbf{d}_a(\zeta_i) = (c_a(\zeta_i) - c_0(\zeta_i))\mathbf{x}(\zeta_i)$, so $\mathbf{x}(\zeta_i)$ is recovered as soon as $c_a(\zeta_i) \neq c_0(\zeta_i)$ for some $a \in [m]$. Conditioned on c_0, each c_a hits the value $c_0(\zeta_i)$ with probability at most $\frac{1}{2}$, independently for $a = 1, \ldots, m$, so slot i fails with probability at most 2^{-m}, and some slot fails with probability at most $\varphi 2^{-m}$. Otherwise the adversary knows $\mathbf{x}$, computes $\mathbf{s}_h = \gamma^{-1}\mathbf{x}$, and interpolates the sharing polynomial at 0 from $\mathbf{s}_h$ and the $t-1$ corrupt shares, which gives $\mathbf{s}$. The attack makes one query of the first round and $m+1$ queries of the last. Its algebraic core, $\mathbf{s}_h = ((c-c')\gamma)^{-1}(\mathbf{z} - \mathbf{z}')$ when $(c-c')\gamma$ is a unit, is the lemma `replay_recovers_share` of our EasyCrypt development. □

Bounded state of Thimza**.** A token of Thimza is created by Sign_1, moves to phase 2 in a successful Sign_2, and is erased in the successful Sign_3 that consumes it, while failed calls create no state. The signer keeps no other information between calls besides its key, so Thimza has bounded state in the sense of Definition 7. Its masks are $\mathbf{m}_i(\mathsf{vw}) = \sum_{j\in T\setminus\{i\}} \varepsilon_{ij}\mathsf{PRF}(\mathsf{sd}_{\{i,j\}}, \mathsf{vw})$ with $\varepsilon_{ij} = -\varepsilon_{ji}$, so $\sum_{i\in T} \mathbf{m}_i(\mathsf{vw}) = 0$ for every view, and Thimza has linear responses with $\gamma_i = \lambda_{T,i}$ and $f_i = \mathbf{r}_i$, which is the case of Theorem 3. Its security theorems hold in this model, so its state is the least that a scheme with linear responses can keep.

Appendix B. Proof of Theorem 8

We prove Theorem 8 with the games $G_0, \ldots, G_9$ of Section 5, where G_0 is the game $\mathsf{TS\text{-}UF\text{-}4[o]}$ of Definition 4 played with Thimza and $\Pr[G_i]$ is the probability that the adversary wins G_i.

Conventions. A call of a signing oracle evaluates H_0 at most $T_{\max}+1$ times, H_1 at most once and H_c at most once, and the verification of the forgery evaluates H_c once, so the adversary and the challenger together query H_0, H_1 and H_c at most Q_0, Q_1 and Q_c times. In the opening semantics a complete request V (Definition 8) has $|S_2(V)| = |S_4(V)| = t - |T \cap \mathcal{CS}| \geq t - |\mathcal{CS}|$, so a forgery on its message is trivial. Each complete request has a unique last opener, and a token opens at most one request, so there are at most Q_s complete requests and at most $L = Q_s T_{\max}$ corrupt first messages in them.

Regularity. If $\mathbf{A}$ is good for the regularity lemma, which fails with probability at most $2q^{-\varphi}$, then for every fixed $\mathbf{Y} \in \mathcal{R}_q^k$ the values $\lfloor \mathbf{Ar}+\mathbf{e}' \rceil_{\nu_c}$ and $\lfloor 2^{\nu_c}\lfloor \mathbf{Ar}+\mathbf{e}' \rceil_{\nu_c} + \mathbf{Y} \rceil_{\nu_w}$, for $(\mathbf{r}, \mathbf{e}') \leftarrow D_{\sigma_w}^{\ell} \times D_{\sigma_w}^{k}$, take each value with probability at most $2^{-\mu}$. $\sqrt{2\pi}\sigma_w \geq 2^{38.3}$ while the bound of the lemma is at most $2^{32.8}$ for our parameter sets, so $\mathbf{Ar} + \mathbf{e}'$ is within statistical distance $2^{-\varphi}$ of uniform. A value of $\lfloor \cdot \rceil_{\nu_c}$ has at most $2^{\nu_c} + 1$ preimages in each coefficient. A value of $\lfloor 2^{\nu_c}\lfloor \cdot \rceil_{\nu_c} + \mathbf{Y} \rceil_{\nu_w}$ fixes each coefficient of $2^{\nu_c}\lfloor \cdot \rceil_{\nu_c}$ to an interval of length at most $2^{\nu_w} + 1$, which contains at most $2^{\nu_w - \nu_c} + 2$ multiples of 2^{ν_c}, so it has at most $(2^{\nu_w-\nu_c} + 2)(2^{\nu_c} + 1) \leq 2^{\nu_w+1}$ preimages in each coefficient. As $2^{\nu_w+1} \leq q2^{-1/k}$, each value has at most $q^{k\varphi}2^{-\varphi}$ preimages in $\mathcal{R}_q^k$, and hence probability at most $2^{-\varphi} + 2^{-\varphi} = 2^{-\mu}$.

Simulation. Every reduction below simulates the random oracles for a quantum adversary with functions that are independent on any $2Q$ inputs [54], which is perfect for Q queries, keeps the programmed points of an oracle in a classical list that it checks on each query, and samples $\mathbf{A}$ through Expand programmed at its seed before the adversary starts, which is also perfect. No reduction rewinds the adversary or records its queries, and every change below is local to one token or one request, so the argument holds for any number of concurrent sessions and any interleaving of their rounds.

G_1**.** For every pair $\{i, j\} \subseteq \mathcal{H}$, the values $\mathsf{PRF}(\mathsf{sd}_{\{i,j\}}, \cdot)$ are replaced by a random function $\mathsf{RF}_{\{i,j\}}$, which the challenger samples lazily and queries classically. A distinguisher $\mathcal{B}_1$ that runs the game with its oracles gives $|\Pr[G_0] - \Pr[G_1]| \leq \mathsf{Adv}^{\mathsf{PRF}}_{\mathcal{B}_1}(\binom{n}{2})$.

G_2**.** The game aborts if two distinct requests opened in $\mathcal{O}_2$ have the same digest, or if $\mathcal{O}_3$ accepts, at an index j whose entry in the request is the first message of a token u of the honest party j, an opening other than the truncated commitment of u. In the first case the two requests are distinct inputs of H_1 with equal outputs. In the second case, the accepted opening with the message of the request and the truncated commitment of u with the message of u are distinct inputs of H_0 with equal outputs. The challenger evaluates all these inputs classically, so an algorithm that runs G_1 and outputs them makes at most Q_0 queries to H_0 and Q_1 to H_1, and the collision bound gives $|\Pr[G_1] - \Pr[G_2]| \leq 27((Q_0+2)^3 + (Q_1+2)^3)/2^{n_H}$.

G_3**.** We split H_0 into its restriction $H_0^{\mathcal{H}}$ to inputs with an honest index and its restriction $H_0^{\mathcal{CS}}$ to the other inputs, which are independent random functions, so nothing changes. Then every query to $H_0^{\mathcal{CS}}$, quantum or classical, is answered by the extractable simulator S of [15], and

when a request V becomes complete, the game computes $\hat{x}_j = S.\mathsf{E}(\mathsf{cmt}_j)$ for every $j \in T \cap \mathcal{CS}$. With at most Q_0 queries and L extractions, the first part of the online extraction fact gives $|\Pr[G_2] - \Pr[G_3]| \le 8L(Q_0 + L)\sqrt{2/2^{n_H}}$.

G_4**.** The game aborts if $\mathcal{O}_3$ accepts, for a complete request and a corrupt index j, an opening $\bar{\mathbf{w}}_j$ with $(\mathsf{vk}, \mathsf{msg}, j, \bar{\mathbf{w}}_j) \neq \hat{x}_j$. By the second part of the online extraction fact, $|\Pr[G_3] - \Pr[G_4]| \le 8L(Q_0+1)\sqrt{2/2^{n_H}} + (40e^2(Q_0 + L + 1)^3 + 2)/2^{n_H}$. From now on, the openings with which the honest members of a complete request respond are the extracted corrupt openings and the revealed honest ones, so the aggregate commitment $\mathbf{w}(V)$ and the point $x_V = (\mathsf{vk}, \mathbf{w}(V), \mathsf{msg})$ are fixed when V becomes complete.

G_5**.** Honest first messages become uniform values, and a token samples its nonce only when it opens. For every token u the challenger samples a nonce $(\mathbf{r}_u, \mathbf{e}'_u) \leftarrow D^{\ell}_{\sigma_{\mathrm{w}}} \times D^{k}_{\sigma_{\mathrm{w}}}$ with truncated commitment $\bar{\mathbf{w}}_u$, and a uniform function F_u from pairs (msg', i') to $\{0,1\}^{n_H}$. In $\mathcal{O}_1(i, \mathsf{msg})$ it returns $\mathsf{cmt}_u = F_u(\mathsf{msg}, i)$ without querying H_0, and from the $\mathcal{O}_2$ call in which u opens on, it answers every query to $H_0^{\mathcal{H}}$ at a point of the set $S_u = \{(\mathsf{vk}, \mathsf{msg}', i', \bar{\mathbf{w}}_u)\}$ with $F_u(\mathsf{msg}', i')$ instead of querying $H_0^{\mathcal{H}}$ there. Since the nonce is not used before that call, the challenger may as well sample it then. Let $S = \bigcup_u S_u$, and let G equal $H_0^{\mathcal{H}}$ outside S and F_u on S_u. If the $\bar{\mathbf{w}}_u$ are distinct, which fails with probability at most $Q_s^2 2^{-\mu}$, the sets S_u are disjoint and G is a uniform function. The game run with G in place of $H_0^{\mathcal{H}}$ is G_4, because $\mathsf{cmt}_u = G(\mathsf{vk}, \mathsf{msg}, i, \bar{\mathbf{w}}_u)$ and the answers on S_u agree with G, and the game run with $H_0^{\mathcal{H}}$ is G_5. Before u opens, $\bar{\mathbf{w}}_u$ is independent of the run and takes each value with probability at most $2^{-\mu}$, and after it opens no query is made in S_u, so a measured query lies in S with probability at most $Q_s 2^{-\mu}$. The hidden point fact, applied to the game as one algorithm with at most Q_0 queries, bounds the change by $2Q_0\sqrt{Q_s 2^{-\mu}} + Q_s^2 2^{-\mu}$.

The game G_5 also aborts in two cases, which we add to G_4 before the change of oracle. The first case is that $\mathcal{O}_3$ accepts an opening at an entry that is the first message of a token u which has not opened. By G_2 such a check queries $H_0^{\mathcal{H}}$ at the point $(\mathsf{vk}, \mathsf{msg}, i, \bar{\mathbf{w}}_u)$ of u itself. Let $\mathcal{B}$ run the game, open every token at the end, and output whether some check queried the point of a token that had not opened. With lazily sampled nonces, the nonce of such a token is drawn after the check, so $\mathcal{B}$ outputs one with probability at most $Q_0 Q_s 2^{-\mu}$, and the hidden point fact applied to $\mathcal{B}$ bounds this abort by $2Q_0\sqrt{Q_s 2^{-\mu}} + Q_s^2 2^{-\mu} + Q_0 Q_s 2^{-\mu}$. The second case is that two tokens get the same first message, which has probability at most $Q_s^2 2^{-n_H}$ since the first messages are independent and uniform. Hence $|\Pr[G_4] - \Pr[G_5]| \le 4Q_0\sqrt{Q_s 2^{-\mu}} + 2Q_s^2 2^{-\mu} + Q_0 Q_s 2^{-\mu} + Q_s^2 2^{-n_H}$. The first abort guarantees that every honest response under a request uses openings that agree with those of its honest members, and the second that two tokens of one party never open the same request, so each honest member responds at most once under a request.

G_6**.** When the token u becomes the last opener of a complete request V, it takes as nonce a fresh pair $(\tilde{\mathbf{r}}_u, \tilde{\mathbf{e}}'_u) \leftarrow D^{\ell}_{\sigma_{\mathrm{w}}} \times D^{k}_{\sigma_{\mathrm{w}}}$, the noise of a hint, which has the distribution of an ordinary nonce, so nothing changes. Then, inside the same $\mathcal{O}_2$ call and before $\bar{\mathbf{w}}_u$ is returned, the challenger reprograms $H_{\mathrm{c}}(x_V)$ to a fresh uniform challenge $c_u \in \mathcal{C}$, aborting if x_V was programmed before, and sets $\mathbf{y}_u = c_u \mathbf{s} + \tilde{\mathbf{r}}_u$ and $\mathbf{y}'_u = c_u \mathbf{e} + \tilde{\mathbf{e}}'_u$, so that $(c_u, \mathbf{y}_u, \mathbf{y}'_u)$ is distributed as a hint of $\mathsf{HintMLWE}$. Given everything before the call, x_V is a function of the fresh nonce, of the revealed honest openings and of the extracted corrupt openings, so it takes each value with probability at most $2^{-\mu}$. The adversary learns x_V only after the reprogramming, and without the reprogramming the game is G_5, since the response then uses the challenge $H_{\mathrm{c}}(x_V)$. Adaptive reprogramming with at most Q_s reprogrammings and Q_{c} queries, and a union bound over earlier programmed points, give $|\Pr[G_5] - \Pr[G_6]| \le Q_s(\sqrt{Q_{\mathrm{c}} 2^{-\mu}} + Q_{\mathrm{c}} 2^{-\mu-1}) + Q_s^2 2^{-\mu}$. By G_4, every honest response under V uses the challenge c_u.

G_7**.** The challenger answers $\mathcal{O}_3$ without the honest shares and masks. Fix a request V. By G_1 the pair values $\mathsf{RF}_{\{i,j\}}(\mathsf{vw}(V))$ of honest pairs are uniform, and by G_2 no other request has the digest $\mathsf{vw}(V)$, so these values occur only in responses under V, and responses under distinct requests use independent values. By G_5 each honest member responds at most once under V, and Lemma 10, applied to each request, shows that a response by a member that is not the last member of $\mathcal{H} \cap T$ to respond under V is uniform and independent of everything else, and that the last response, if all of $\mathcal{H} \cap T$ respond, is $c_u \sum_{h \in \mathcal{H} \cap T} \lambda_{T,h} \mathbf{s}_h + \sum_{h \in \mathcal{H} \cap T} \mathbf{r}_h + \mathbf{m}_{\mathcal{CS}}(V)$ minus the earlier ones. A request whose honest members all respond is complete, because responding requires opening, so its last opener u used the nonce $(\tilde{\mathbf{r}}_u, \tilde{\mathbf{e}}'_u)$ and $c_u = H_{\mathrm{c}}(x_V)$. A request whose entry at some honest index is not a first message of that party, for instance one that the adversary computed itself, is never opened by that party, so all its responses are uniform. By Lagrange reconstruction, $c_u \sum_h \lambda_{T,h} \mathbf{s}_h = c_u \mathbf{s} - c_u \sum_{j \in T \cap \mathcal{CS}} \lambda_{T,j} \mathbf{s}_j$, and the last response is

$$\mathbf{y}_u + \textstyle\sum_{h \neq u} \mathbf{r}_h - c_u \sum_{j \in T \cap \mathcal{CS}} \lambda_{T,j} \mathbf{s}_j + \mathbf{m}_{\mathcal{CS}}(V) - \sum \mathbf{z}_{\text{earlier}}, \tag{4}$$

where h runs over the other honest members of T, while the opening of u is $\lfloor \mathbf{A}\mathbf{y}_u + \mathbf{y}'_u - c_u \mathbf{b} \rceil_{\nu_{\mathrm{c}}}$ with $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$, since $\mathbf{A}\mathbf{y}_u + \mathbf{y}'_u - c_u \mathbf{b} = \mathbf{A}\tilde{\mathbf{r}}_u + \tilde{\mathbf{e}}'_u$. The challenger computes uniform responses and (4), which gives the same distribution, so $\Pr[G_6] = \Pr[G_7]$. The two identities are the lemmas `honest_aggregate` and `hint_commit` of our `EasyCrypt` development.

G_8**.** The corrupt shares are sampled uniformly. Write $\mathbf{s}_j - \mathbf{s} = \sum_{v=1}^{t-1} \alpha_j^v \mathbf{p}_v$ for $j \in \mathcal{CS}$. Since $|\mathcal{CS}| \le t - 1$, the matrix $(\alpha_j^v)_{j \in \mathcal{CS},\, v \le |\mathcal{CS}|}$ is a Vandermonde matrix whose rows are scaled by the units α_j, with unit differences, so it is invertible over $\mathcal{R}_q$, and the map from $(\mathbf{p}_v)_v$ to $(\mathbf{s}_j - \mathbf{s})_{j \in \mathcal{CS}}$ is onto. Hence the corrupt shares are uniform and independent of $\mathbf{s}$, and since G_7 no longer uses the honest shares, $\Pr[G_7] = \Pr[G_8]$. In G_8 the secret $(\mathbf{s}, \mathbf{e})$ is used only

through $\mathbf{b}$ and the triples $(c_u, \mathbf{y}_u, \mathbf{y}'_u)$, which are at most Q_s hints of HintMLWE with $\sigma_1 = \sigma_{\mathrm{t}}$ and $\sigma_2 = \sigma_{\mathrm{w}}$.

G_9**.** The vector $\mathbf{b}$ is uniform in $\mathcal{R}_q^k$ and $\mathbf{t} = \lfloor \mathbf{b} \rceil_{\nu_{\mathrm{t}}}$. The reduction $\mathcal{B}_2$ receives $(\mathbf{A}, \mathbf{b})$ and Q_s hints. It runs G_8 with $\mathbf{t} = \lfloor \mathbf{b} \rceil_{\nu_{\mathrm{t}}}$, gives the hints to the last openers in the order in which they open, programs $H_{\mathrm{c}}(x_V)$ to the challenge of that hint, computes the opening and (4) from the hint, and outputs whether the adversary wins. This is G_8 when $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$ and G_9 when $\mathbf{b}$ is uniform, so $|\Pr[G_8] - \Pr[G_9]| \leq \mathsf{Adv}^{\mathsf{HintMLWE}_{q,k,\ell,Q_s,\sigma_{\mathrm{t}},\sigma_{\mathrm{w}}}}_{\mathcal{B}_2}$.

Final step. Let the adversary win G_9 with $(\mathsf{msg}^*, (c^*, \mathbf{z}^*, \mathbf{h}^*))$ and let $\mathbf{w}^* = \lfloor \mathbf{A}\mathbf{z}^* - 2^{\nu_{\mathrm{t}}} c^* \mathbf{t} \rceil_{\nu_{\mathrm{w}}} + \mathbf{h}^*$, so that verification gives $c^* = H_{\mathrm{c}}(\mathsf{vk}, \mathbf{w}^*, \mathsf{msg}^*)$. If $(\mathsf{vk}, \mathbf{w}^*, \mathsf{msg}^*)$ were a programmed point x_V, then V would be a complete request for msg^*, and the forgery would be trivial, so the point is not programmed. The reduction $\mathcal{B}_3$ receives $(\mathbf{A}, \mathbf{b})$ and quantum access to G. It samples $(\mathbf{s}, \mathbf{e})$ and the hints itself, which is possible because $\mathbf{b}$ is independent of them in G_9, answers $H_{\mathrm{c}}(\mathsf{vk}', \mathbf{w}, \mathsf{msg}')$ with $G(2^{\nu_{\mathrm{w}}}\mathbf{w}, (\mathsf{vk}', \mathsf{msg}'))$ except at the programmed points, which it keeps in a classical list, runs G_9, and outputs the string $\mathsf{aux} = (\mathsf{vk}, \mathsf{msg}^*)$ and the vector $(\mathbf{z}^*, c^*, \mathbf{u}^*)$ with $\mathbf{u}^* = 2^{\nu_{\mathrm{w}}}\mathbf{w}^* - (\mathbf{A}\mathbf{z}^* - 2^{\nu_{\mathrm{t}}} c^* \mathbf{t})$. Since $\mathbf{w} \mapsto 2^{\nu_{\mathrm{w}}}\mathbf{w}$ is injective on $\mathbb{Z}_{q_{\nu_{\mathrm{w}}}}$, the simulated H_{c} is a uniform function. The image of the solution under $[\mathbf{A} \mid -2^{\nu_{\mathrm{t}}}\mathbf{t} \mid \mathbf{I}]$ is $2^{\nu_{\mathrm{w}}}\mathbf{w}^*$, which G maps to c^* with second argument aux. The vector $\mathbf{u}^* - 2^{\nu_{\mathrm{w}}}\mathbf{h}^*$ is the residue of rounding $\mathbf{A}\mathbf{z}^* - 2^{\nu_{\mathrm{t}}} c^* \mathbf{t}$ to ν_{w} bits, plus at most one wrap around modulo $q_{\nu_{\mathrm{w}}}$, and $q = q_{\nu_{\mathrm{w}}} 2^{\nu_{\mathrm{w}}} + 1$, so each of its coefficients is at most $2^{\nu_{\mathrm{w}}-1} + 2$ in absolute value. Hence $\|(\mathbf{z}^*, c^*, \mathbf{u}^*)\|_2 \leq \|(\mathbf{z}^*, 2^{\nu_{\mathrm{w}}}\mathbf{h}^*)\|_2 + (2^{\nu_{\mathrm{w}}-1} + 2)\sqrt{k\varphi} + \sqrt{\omega} \leq B + (2^{\nu_{\mathrm{w}}-1} + 2)\sqrt{k\varphi} + \sqrt{\omega} = \beta$, and $\Pr[G_9] \leq \mathsf{Adv}^{\mathsf{SelfTargetMSIS}_{q,k,\ell,\mathcal{C},\beta}}_{\mathcal{B}_3}$, which is the lemma `final_reduction` of our EasyCrypt development.

Summing the hops. We add the bounds of all hops. The distinct point terms of G_5 and G_6 add up to $2Q_s^2 2^{-\mu} + Q_s^2 2^{-\mu} \leq 5Q_s^2 2^{-\mu}$, the term $Q_s^2 2^{-n_H}$ of G_5 is added to the collision term, and the regularity lemma fails with probability at most $2q^{-\varphi}$. This gives $\mathsf{Adv}^{\mathsf{TS\text{-}UF\text{-}4[o]}}_{\mathcal{A}} \leq \mathsf{Adv}^{\mathsf{PRF}}_{\mathcal{B}_1}(\binom{n}{2}) + \mathsf{Adv}^{\mathsf{HintMLWE}}_{\mathcal{B}_2} + \mathsf{Adv}^{\mathsf{SelfTargetMSIS}}_{\mathcal{B}_3} + \epsilon$. □

Appendix C. Proof of Theorem 9

In the response semantics, a request V is *answered* once every member of $\mathcal{H} \cap T$ has responded under V. An answered request is complete, since responding requires opening, and a forgery on its message is trivial. We define G_0 to G_8 as in Appendix B, starting from $\mathsf{TS\text{-}UF\text{-}4[r]}$. Every hop bounds the change of the distribution of the whole run, including the logs and the forgery, or is a distinguisher that evaluates the winning condition itself, so $|\Pr[G_0] - \Pr[G_8]| \leq \mathsf{Adv}^{\mathsf{PRF}}_{\mathcal{B}_1}(\binom{n}{2}) + \epsilon$.

Partition. Number the complete requests of G_8 as $V_1, V_2, \ldots$ in the order in which they become complete, at most Q_s of them, with points $x_g = x_{V_g}$, distinct by the abort of G_6. For a forgery $(\mathsf{msg}^*, (c^*, \mathbf{z}^*, \mathbf{h}^*))$ let $x^* = (\mathsf{vk}, \lfloor \mathbf{A}\mathbf{z}^* - 2^{\nu_{\mathrm{t}}} c^* \mathbf{t} \rceil_{\nu_{\mathrm{w}}} + \mathbf{h}^*, \mathsf{msg}^*)$. Let E_g, for $1 \leq g \leq Q_s$, be the event that V_g exists, the adversary wins and $x^* = x_g$, and let E_0 be the event that it wins and x^* is no x_g, so $\Pr[G_8] = \sum_{g=0}^{Q_s} \Pr[E_g \text{ in } G_8]$. On E_g with $g \geq 1$ the message msg^* is that of V_g, so V_g is not answered and $\mathcal{O}_3$ never returns the last response under V_g.

Unprogramming x_g**.** For $1 \leq g \leq Q_s$, the game $G_8^{(g)}$ differs from G_8 only at V_g: its last opener u takes an ordinary nonce and no hint, $H_{\mathrm{c}}(x_g)$ keeps its value, although x_g still counts for the abort of G_6, and the game aborts when $\mathcal{O}_3$ would return the last response under V_g. Let G'_8 be G_8 with this last abort, which E_g excludes, so $\Pr[E_g \text{ in } G_8] = \Pr[E_g \text{ in } G'_8]$. In G'_8 the other responses under V_g are uniform by G_7, and the hint $(c_u, \mathbf{y}_u, \mathbf{y}'_u)$ of u enters only the last response under V_g, which is never computed, and the opening of u, which is that of the fresh nonce $(\tilde{\mathbf{r}}_u, \tilde{\mathbf{e}}'_u)$ since $\mathbf{A}\mathbf{y}_u + \mathbf{y}'_u - c_u\mathbf{b} = \mathbf{A}\tilde{\mathbf{r}}_u + \tilde{\mathbf{e}}'_u$ for $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$. So c_u is used only to program $H_{\mathrm{c}}(x_g)$, and G'_8 and $G_8^{(g)}$ differ by one reprogramming to a fresh value, at a point that, as in G_6, takes each value with probability at most $2^{-\mu}$ and is learned only afterwards. Adaptive reprogramming gives $\Pr[E_g \text{ in } G_8] \leq \Pr[E_g \text{ in } G_8^{(g)}] + \delta$ with $\delta = \sqrt{Q_{\mathrm{c}} 2^{-\mu}} + Q_{\mathrm{c}} 2^{-\mu-1}$. This step needs $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$: for a uniform $\mathbf{b}$ the opening computed from the hint would be that of $\mathbf{A}\tilde{\mathbf{r}}_u + \tilde{\mathbf{e}}'_u + c_u(\mathbf{A}\mathbf{s} + \mathbf{e} - \mathbf{b})$, which depends on c_u. Let $G_8^{(0)} = G_8$.

Reductions. For $0 \leq g \leq Q_s$, let $\mathcal{B}_2^{(g)}$ run $G_8^{(g)}$ as the reduction to HintMLWE of Appendix B runs G_8, with no hint for V_g, and output whether E_g occurs, and let $G_9^{(g)}$ be the game it runs on a uniform $\mathbf{b}$, so that $G_9^{(0)} = G_9$. The reduction $\mathcal{B}_2$ runs $\mathcal{B}_2^{(g)}$ for a uniform $g \in \{0, \ldots, Q_s\}$, and averaging over g gives $\sum_{g=0}^{Q_s}(\Pr[E_g \text{ in } G_8^{(g)}] - \Pr[E_g \text{ in } G_9^{(g)}]) \leq (Q_s + 1)\mathsf{Adv}^{\mathsf{HintMLWE}}_{\mathcal{B}_2}$. The reduction $\mathcal{B}_4$ receives $(\mathbf{A}, \mathbf{b})$ and quantum access to G, and runs $G_9^{(g)}$ for a uniform $g \in \{0, \ldots, Q_s\}$ as $\mathcal{B}_3$ runs G_9 in Appendix B: it samples $(\mathbf{s}, \mathbf{e})$ and the hints itself, which is possible because $\mathbf{b}$ is independent of them, and programs the points of the complete requests other than V_g. On E_g the point x^* is not programmed, by definition for $g = 0$ and by the abort of G_6 for $g \geq 1$, so $H_{\mathrm{c}}(x^*)$ is given by G and the final step of Appendix B yields a solution of $\mathsf{SelfTargetMSIS}_{q,k,\ell,\mathcal{C},\beta}$. Averaging over g gives $\sum_{g=0}^{Q_s} \Pr[E_g \text{ in } G_9^{(g)}] \leq (Q_s + 1)\mathsf{Adv}^{\mathsf{SelfTargetMSIS}}_{\mathcal{B}_4}$.

Combining the three bounds, $\Pr[G_8] \leq (Q_s + 1)(\mathsf{Adv}^{\mathsf{HintMLWE}}_{\mathcal{B}_2} + \mathsf{Adv}^{\mathsf{SelfTargetMSIS}}_{\mathcal{B}_4}) + Q_s\delta$, and adding the bound on $|\Pr[G_0] - \Pr[G_8]|$ proves the theorem. □

Appendix D. Proof of Theorem 5

Rounding residues. Because $q = q_\nu 2^\nu + 1$ for $\nu \leq 41$, every $x \in \mathbb{Z}_q$ with representative in $[0, q)$ satisfies $x + 2^{\nu-1} = 2^\nu \lfloor (x + 2^{\nu-1})/2^\nu \rfloor + r$ with $0 \leq r < 2^\nu$, and the reduction modulo q_ν changes the quotient only when it equals q_ν, which happens for $x \geq q - 2^{\nu-1}$ and moves $2^\nu \lfloor x \rceil_\nu$ by $2^\nu q_\nu = q - 1 \equiv -1$. Hence $2^\nu \lfloor x \rceil_\nu = x + \epsilon$ in $\mathbb{Z}_q$ with $|\epsilon| \leq 2^{\nu-1} + 1$.

Exact identities. All members of T hold tokens with the same digest, so their masks cancel, and $\sum_{i \in T} \lambda_{T,i}\mathbf{s}_i = \mathbf{s}$

by Lagrange interpolation at 0, so $\mathbf{z} = \sum_i \mathbf{z}_i = c\mathbf{s} + \sum_j \mathbf{r}_j$. Each opening satisfies $2^{\nu_c}\bar{\mathbf{w}}_j = \mathbf{A}\mathbf{r}_j + \mathbf{e}'_j + \delta_j$ with a truncation residue $\|\delta_j\|_\infty \le 2^{\nu_c - 1} + 1$, so the sum of the scaled openings is $\bar{\mathbf{W}} = \sum_j 2^{\nu_c}\bar{\mathbf{w}}_j = \sum_j (\mathbf{A}\mathbf{r}_j + \mathbf{e}'_j) + \delta$ with $\delta = \sum_j \delta_j$. The key satisfies $2^{\nu_t}\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e} + \rho$ with a key residue $\|\rho\|_\infty \le 2^{\nu_t - 1} + 1$. Then $\mathbf{A}\mathbf{z} - 2^{\nu_t} c\mathbf{t} = \mathbf{W} - \mathbf{u}$ with $\mathbf{u} = \sum_j \mathbf{e}'_j + \delta + c(\mathbf{e} + \rho)$. The signers and the combiner compute $\mathbf{w} = \lfloor \mathbf{W} \rceil_{\nu_w}$ and $\mathbf{h} = \mathbf{w} - \lfloor \mathbf{W} - \mathbf{u} \rceil_{\nu_w}$, so the verifier recomputes $\lfloor \mathbf{A}\mathbf{z} - 2^{\nu_t} c\mathbf{t} \rceil_{\nu_w} + \mathbf{h} = \mathbf{w}$ and the challenge $c = H_c(\mathsf{vk}, \mathbf{w}, \mathsf{msg})$. Writing $2^{\nu_w}\mathbf{w} = \mathbf{W} + \epsilon_1$ and $2^{\nu_w}\lfloor \mathbf{W} - \mathbf{u} \rceil_{\nu_w} = \mathbf{W} - \mathbf{u} + \epsilon_2$ gives $2^{\nu_w}\mathbf{h} = \mathbf{u} + \epsilon_1 - \epsilon_2$, where $\epsilon = \epsilon_1 - \epsilon_2$ has $\|\epsilon\|_\infty \le 2^{\nu_w} + 2$. The centred representative of $\mathbf{h}$ modulo q_{ν_w} adds at most one more wrap, which is absorbed in the same bound because $\|\mathbf{u} + \epsilon\|_\infty$ is far below $q/2$. Thus Verify accepts if and only if the norm condition holds. These identities are the lemmas `correctness`, `resp_sum`, `hint_correct` and `combine_hash` of our EasyCrypt development.

Variance. We model the residues δ_j, ρ, ϵ_1 and ϵ_2 as independent and uniform on their intervals, of variance $4^\nu/12$ per coefficient for $\nu = \nu_c, \nu_t, \nu_w, \nu_w$, as in the analyses of Raccoon and Threshold Raccoon [4], [5]. A coefficient of $c\mathbf{s}$ is a sum of ω terms $\pm s$ with s of variance σ_t^2, so it has variance $\omega\sigma_t^2$, and likewise $c\mathbf{e}$ has variance $\omega\sigma_t^2$ and $c\rho$ has variance $\omega 4^{\nu_t}/12$. Hence a coordinate of $\mathbf{z}$ has variance $t\sigma_w^2 + \omega\sigma_t^2$, and a coordinate of $2^{\nu_w}\mathbf{h}$ has variance $t\sigma_w^2 + t4^{\nu_c}/12 + \omega\sigma_t^2 + \omega 4^{\nu_t}/12 + 4^{\nu_w}/6$, and both are at most v for $t \le T_{\max}$.

Tail. Laurent and Massart [58] show that $X = \sum_{i=1}^{d} a_i Z_i^2$ with independent standard normal variables Z_i and weights $a_i \ge 0$ satisfies $\Pr[X \ge \sum_i a_i + 2\sqrt{\tau \sum_i a_i^2} + 2\tau \max_i a_i] \le e^{-\tau}$. With $a_i \le v$ the threshold is at most $v(d + 2\sqrt{d\tau} + 2\tau) = B^2$, so modelling the $d = (\ell + k)\varphi$ coordinates as independent centred Gaussians of variance at most v gives $\Pr[\|(\mathbf{z}, 2^{\nu_w}\mathbf{h})\|_2 > B] \le e^{-\tau} = 2^{-\lambda}$ for $\tau = \lambda \ln 2$. Section 6 checks the model against the exact distributions of the protocol. □

Appendix E. Proofs of Theorems 6 and 7

Interchangeability. The verifier of Thimza is the predicate (1) with the bound B of the parameter set, which mentions neither t nor T, and it is the Raccoon verifier (lemma `verify_is_raccoon`). For $t = n = 1$, the sharing polynomial is constant, so $\mathbf{s}_1 = \mathbf{s}$ and $\lambda_{\{1\},1} = 1$, the signer set has no other member, so the mask is an empty sum, and the request contains one first message, which the signer checks against its own. The three rounds then compute $\bar{\mathbf{w}} = \lfloor \mathbf{A}\mathbf{r} + \mathbf{e}' \rceil_{\nu_c}$, $\mathbf{w} = \lfloor 2^{\nu_c}\bar{\mathbf{w}} \rceil_{\nu_w}$, $c = H_c(\mathsf{vk}, \mathbf{w}, \mathsf{msg})$ and $\mathbf{z} = c\mathbf{s} + \mathbf{r}$, after which Combine computes the Raccoon hint. This is Raccoon signing with the commitment $2^{\nu_c}\lfloor \mathbf{A}\mathbf{r} + \mathbf{e}' \rceil_{\nu_c}$, which is the lemma `single_signer`, and the lemma `single_signer_verifies` states that the result is accepted. For every t, Theorem 5 shows that an honest signature is accepted under its norm condition. □

Privacy of the signer set. Fix the key, the message and two signer sets T_0, T_1 of size t, and consider honest executions in which all members receive the same request. By the exact identities of Appendix D, the output of Combine with the signer set T_b is $(c, c\mathbf{s} + \mathbf{R}, \mathbf{h})$ with the nonce sum $\mathbf{R} = \sum_{j \in T_b} \mathbf{r}_j$, $c = H_c(\mathsf{vk}, \lfloor \mathbf{W} \rceil_{\nu_w}, \mathsf{msg})$ for $\mathbf{W} = \sum_{j \in T_b} 2^{\nu_c}\lfloor \mathbf{A}\mathbf{r}_j + \mathbf{e}'_j \rceil_{\nu_c}$, and $\mathbf{h} = \lfloor \mathbf{W} \rceil_{\nu_w} - \lfloor \mathbf{A}(c\mathbf{s} + \mathbf{R}) - 2^{\nu_t} c\mathbf{t} \rceil_{\nu_w}$. It is therefore a fixed function of the pair $(\mathbf{R}, \mathbf{W})$, which is the sum over $j \in T_b$ of t independent pairs $(\mathbf{r}_j, 2^{\nu_c}\lfloor \mathbf{A}\mathbf{r}_j + \mathbf{e}'_j \rceil_{\nu_c})$ with the same distribution. The distribution of such a sum depends on t only, so the output has the same distribution for T_0 and T_1. The lemma `signer_set_privacy` proves this as an equivalence of the two executions, which pass every check of the three rounds (`honest_ok2`, `honest_ok3`). □

Appendix F. Unforgeability of Raccoon

Theorem 8 also covers single signer Raccoon. Corollary 11 states this guarantee.

Corollary 11. *Raccoon signing with the commitment $2^{\nu_c}\lfloor \mathbf{A}\mathbf{r} + \mathbf{e}' \rceil_{\nu_c}$, and the verifier* (1) *with the parameters of Table 3, is existentially unforgeable under chosen message attacks in the quantum random oracle model. An adversary that makes Q signing queries has advantage at most $\mathsf{Adv}^{\mathsf{HintMLWE}_{q,k,\ell,Q,\sigma_t,\sigma_w}} + \mathsf{Adv}^{\mathsf{SelfTargetMSIS}_{q,k,\ell,\mathcal{C},\beta}} + \epsilon$, where ϵ is evaluated with $Q_s = 3Q$, $T_{\max} = 1$ and $L = 0$.*

Proof. An adversary against the single signer scheme is turned into an adversary against TS-UF-0 of Thimza with $t = n = 1$ and $\mathcal{CS} = \emptyset$, which answers each signing query on msg by calling $\mathcal{O}_1(1, \mathsf{msg})$, then $\mathcal{O}_2$ on the request $(\mathsf{msg}, \{1\}, (\mathsf{cmt}))$ and $\mathcal{O}_3$ with its own opening, and returns the combined signature. By Theorem 6 the answers have the distribution of single signer signatures, and a forgery on a message that was never queried is a forgery in the sense of TS-UF-0. Lemma 2 and Theorem 8 bound its advantage. With $n = 1$ there are no honest pairs, so the PRF term vanishes, and with $\mathcal{CS} = \emptyset$ there are no corrupt first messages to extract, so $L = 0$ and the extraction terms vanish. The same argument with $\nu_c = 0$, in which the opening is the commitment itself, applies to Raccoon signing without truncation. □

Appendix G. Parameter Derivation

Table 10 lists the quantities from which Table 3 is derived, at the three levels. The modulus $q = 2^{49} - 2^{41} + 1$ is prime and satisfies $q \equiv 1 \bmod 2^{41}$, hence $q \equiv 1 \bmod 2\varphi$, and its first row gives $\log_2 q$. The second row gives $\eta = \sqrt{\ln(2\varphi(1 + 2^\lambda))/\pi}$, an upper bound on the smoothing parameter $\eta_{2^{-\lambda}}(\mathbb{Z}^\varphi)$ [50]. The next rows give the largest budget Q_s with $\sigma_t^{-2} + Q_s\omega^2\sigma_w^{-2} \le \pi/(2\eta^2)$, the chosen budget, the width σ_0 that Lemma 1 gives for the chosen budget, the base two logarithm of the verification bound B of Theorem 5, and the ratio β/q for the bound β of Section 5. The hardness of $\mathsf{MLWE}_{q,k,\ell,\sigma_0}$ is the cost $0.292\beta_{\mathrm{BKZ}}$ of the primal attack with the block size β_{BKZ} that the geometric series assumption predicts, minimized over the number of samples, and the hardness of $\mathsf{SelfTargetMSIS}_{q,k,\ell,\mathcal{C},\beta}$ is the cost of finding a vector of norm at most β in the lattice of $[\mathbf{A} \mid -2^{\nu_t}\mathbf{t} \mid \mathbf{I}]$, minimized over the dimension, and the table

gives both block sizes and both costs. The last row gives the ratio $2^{\nu_c}/(\sigma_w\sqrt{12})$ of the truncation noise to the nonce noise, which is the same at every level. Our implementation of both estimates gives 118.6, 182.2 and 255.2 bits for the three parameter sets of Kyber and 120.9 bits for the second level of Dilithium, against 118, 183, 256 and 123 bits published [59], [62].

TABLE 10. DERIVED QUANTITIES OF THE PARAMETER SETS.

	Level I	Level III	Level V
$\log_2 q$	48.994	48.994	48.994
η	5.518	6.676	7.661
largest $\log_2 Q_s$	61.23	59.27	57.86
chosen $\log_2 Q_s$	60	59	57
σ_0	4.764	4.129	5.818
$\log_2 B$	48.31	48.57	48.71
β/q	0.6703	0.8063	0.9488
MLWE block size	424	720	903
MLWE core SVP	123.8	210.2	263.7
SelfTargetMSIS block size	535	835	984
SelfTargetMSIS core SVP	156.2	243.8	287.3
$2^{\nu_c}/(\sigma_w\sqrt{12})$	0.1443	0.1443	0.1443

Sizes. A verification key is a seed of 32 bytes and $\mathbf{t}$, of $k\varphi(\lceil\log_2 q\rceil - \nu_t)$ bits. The first message has n_H bits, the second $k\varphi(\lceil\log_2 q\rceil - \nu_c)$ bits and the third $\ell\varphi\lceil\log_2 q\rceil$ bits. A signature encodes the challenge by a hash of 2λ bits, as Raccoon does, each coefficient of $\mathbf{z}$ with $\lceil\log_2(\sigma_w\sqrt{T_{\max}})+2.6\rceil$ bits and each coefficient of $\mathbf{h}$ with $\lceil\log_2(\sigma_w\sqrt{T_{\max}2^{-\nu_w}})+2.6\rceil$ bits, which gives the sizes of Table 3.

Appendix H. Additional Measurements

Table 11 gives the signing time in milliseconds of one signer at the 128 bit level for every threshold t from 2 to 1024 that we measured, which is the data of the left plot of Figure 8, and Hermine is measured up to its limit of 64 signers. Thimza is the fastest scheme at every t, from 2.3 ms at $t = 2$ to 60.2 ms at $t = 1024$, and its time grows more slowly with t than that of Threshold Raccoon, since it evaluates $t - 1$ masks against $2t$.

Table 12 gives the kibibytes that one signer sends at the same thresholds, which is the data of the right plot of Figure 8. Thimza sends 16.4 KiB at every t, since none of its messages depends on t, while Threshold Raccoon grows from 39.9 to 55.8 KiB with its authentication tags of 16 bytes per member, TalonG and Hermine send 45.8 and 72.5 KiB, and EKT and Ringtail send more than 270 and 590 KiB. Every scheme produced signatures that its verifier accepted for the five pairs (t, n) of our correctness test.

Appendix I. Measurement on a Satellite Link

This appendix describes the measurement of Section 7.1 in full, for readers who do not work on satellite networks, and it covers the background that the measurement needs, the testbed and the job, the checks of correctness on the node, the method and the results of the signing sessions, the single packets, the time and the space on the node, and the scope of the results. Every number comes from the files that the testbed returned for our job, and the code of the job, its configuration and these files are part of our artifact.

I.1. Satellite Access in Brief

A Starlink user terminal, a flat antenna placed outdoors, exchanges radio signals with satellites that orbit at an altitude of about 550 km [64], and the satellites relay the traffic of the terminal to a ground station, from which it enters the Internet at a point of presence of the operator [65]. The satellites move fast relative to the ground, and the operator recomputes which satellite serves which terminal on a global grid of 15 seconds, at seconds 12, 27, 42 and 57 of every minute of Coordinated Universal Time (UTC) [14], which we call the reconfiguration instants.

Five notions from networking appear below. The round-trip time (RTT) is the time from sending a packet until its answer arrives, and every exchange of messages over a network costs at least one RTT. The maximum transmission unit (MTU) is the largest packet that a path carries, 1500 bytes on this path, and a larger packet is split into IP fragments, which networks often drop [69]. TCP [70] carries a stream of bytes in packets below the MTU, retransmits lost packets and raises its sending rate gradually, so a transfer of a few kilobytes costs about one RTT while a transfer of megabytes is limited by the capacity of the link, and QUIC [71] provides the same over UDP. TLS 1.3 [67] encrypts and authenticates a TCP stream after a handshake of one RTT, and its hybrid group X25519MLKEM768 [68] combines an elliptic curve exchange with ML-KEM-768 [72] in that handshake. Goodput is the rate at which the payload arrives, which is below the capacity of the link because of headers and of the gradual increase of the sending rate. Hence a round of an interactive protocol costs about one RTT plus the time to carry its bytes in both directions, and in Thimza the second term grows with t through the openings that a signer receives.

I.2. The Testbed and the Job

LEOScope [63] consists of nodes, which are computers that universities and volunteers connect to the Internet through a Starlink terminal, and of an orchestrator that schedules jobs on them. A node is not reachable from the Internet, so it contacts the orchestrator periodically and receives the jobs scheduled for it. A job names a container image in a public registry, a configuration file, a node, a start instant and a duration. At the start instant the node downloads the image, creates a directory that holds the configuration file, starts the container as an unprivileged user with this directory at `/artifacts`, stops the container at the end of the duration, and uploads the directory, which the experimenter then downloads. The testbed can also stop a job when the volunteer who hosts a node needs the connection [63], and our job ran to its end.

Our image contains our implementation, its unit tests, the sampler of honest signatures behind Table 4 and a measurement driver, on Debian 13 with Python 3.12.16,

TABLE 11. SIGNING TIME IN MILLISECONDS OF ONE SIGNER AT THE 128 BIT LEVEL.

t	2	4	8	16	32	64	128	256	512	1024
Thimza	2.3	2.4	2.6	3.1	4.0	5.9	9.6	17.0	31.7	60.2
TRaccoon	2.5	2.7	3.2	4.3	6.3	10.4	18.5	34.8	67.4	133.2
TalonG	5.1	5.3	6.3	7.6	10.7	16.8	28.9	53.3	101.6	199.1
Hermine	18.9	19.8	21.2	25.8	32.1	44.4				
EKT	26.1	28.1	30.4	34.7	45.2	69.6	111.5	196.7	370.0	726.3
Ringtail	76.7	80.8	86.1	97.9	119.9	169.2	261.0	450.8	832.2	1598.3

TABLE 12. KIB SENT BY ONE SIGNER AT THE 128 BIT LEVEL.

t	2	4	8	16	32	64	128	256	512	1024
Thimza	16.4	16.4	16.4	16.4	16.4	16.4	16.4	16.4	16.4	16.4
TRaccoon	39.9	39.9	40.0	40.1	40.3	40.8	41.8	43.8	47.8	55.8
TalonG	45.8	45.8	45.8	45.8	45.8	45.8	45.8	45.8	45.8	45.8
Hermine	72.5	72.5	72.5	72.5	72.5	72.5				
EKT	271.9	271.9	271.9	271.9	271.9	271.9	271.9	271.9	271.9	271.9
Ringtail	598.5	598.6	598.6	598.8	599.0	599.5	600.5	602.5	606.5	614.5

TABLE 13. THE NODE AND THE RECORD OF THE RUN.

Property	Value
Node	`starlink-london-ns2`, London
Processor	Intel Core i7-1185G7, 8 logical cores
Clock	up to 4.8 GHz, `powersave` governor
NumPy vector code	up to AVX-512 (Ice Lake set)
Memory	32 GB, no container limit
Host system	Linux 7.0.0-27-generic (Ubuntu)
Container	Debian 13, Python 3.12.16
Libraries	NumPy 2.3.5, OpenSSL 3.5.7
Numerical threads	1
TCP congestion control	CUBIC
Date	5 November 2026
Time (UTC)	09:52:13 to 10:02:30
Load at the start	1.12 over one minute
Stages	correctness 6.3 s, sessions 296.0 s, packets 115.0 s, time 89.9 s, space 97.3 s
Code digest	SHA-256 `c72854c6...ccac3d6`
Session traffic	5.41 MB up, 126.07 MB down
Packet traffic	2.87 MB of echo requests

NumPy 2.3.5 and OpenSSL 3.5.7, and our configuration fixes every parameter that this appendix reports. The driver runs seven stages in a fixed order, each under its own time limit, writes every record to a file in `/artifacts` as soon as it is measured, and records no address and no location of the node. It also records the SHA-256 digest of every source file that it runs, and the digests on the node equal those of our artifact, so the code that ran is the code of Section 7. Table 13 gives the node and the record of the run. Our configuration named no second node, so the stage that runs a coordinator and $t - 1$ real signers on another node of the testbed recorded that it had no server and ended at once; our driver supports this stage for a later run.

I.3. Correctness on the Node

Before any timing, the job checked the implementation on the processor of the node. The 11 unit tests of our implementation passed. The modular multiplication, which uses a floating point quotient, returned the exact product for 20 010 pairs, 10 at the edges of the range and 20 000 at random, against arithmetic on Python integers, and the product of two random ring elements through the number theoretic transform equalled their schoolbook product. At each of the three levels, honest executions with $(t, n) = (2, 3)$, $(3, 5)$, $(5, 8)$ and $(7, 10)$ produced signatures that verify, that still verify after encoding and decoding, and that are rejected for another message and after one coefficient of $\mathbf{z}$ is changed, and the independent verifier of Section 7, which uses schoolbook arithmetic only, accepted a signature and rejected it for another message. The 33 known answers of the exact integer part of the implementation, which cover the expansion of $\mathbf{A}$, the pseudorandom function, the challenge, the transform, H_0, H_1, the Lagrange coefficients, Shamir sharing and the random generator, equal the values that we computed on the M2 laptop, and so do the 15 known answers that depend on the Gaussian sampler, whose floating point functions could in principle differ between processors.

I.4. Method of the Signing Sessions

A session. A session runs one execution of the protocol of Figure 3 with the signer set $T = \{1, \ldots, t\}$, in which the node plays signer 1. It opens a new TCP connection and a new TLS 1.3 connection to the server, sends one empty request whose time serves as the reference round trip of the session, and then runs the three rounds. Each round computes the message of signer 1 with our implementation and sends it in one exchange whose answer carries the broadcast that the coordinator would send next, which is the request $V = (\mathsf{msg}, T, (\mathsf{cmt}_j)_{j \in T})$ after round one, the t openings $(\bar{\mathbf{w}}_j)_{j \in T}$ after round two, and the signature after round three. The node then decodes the signature and checks it with the Raccoon verifier. A session thus uses one connection and seven requests.

The server. The server is the public speed test of Cloudflare [66], which accepts any body on its upload endpoint and returns exactly N bytes on its download endpoint, and every response that names the site of the server, which are the 693 answers to uploads, names London Heathrow. An

exchange writes an HTTP POST that carries the message of the round and an HTTP GET for the size of the broadcast back to back on the same connection, which HTTP/1.1 permits as pipelining [73]. The server answers the requests of a connection in order and reads the whole POST before it answers the GET, so the broadcast leaves the server only after the message of the signer has arrived, as in the protocol, and an exchange costs one round trip plus the two transfers. Before the job, we checked on a terrestrial connection that the server honours pipelining, returns exactly the requested number of bytes without compression, and keeps a connection open between requests.

The bytes. The bodies sent are the exact encodings of the messages of signer 1, which are the 80, 120 or 160 bytes of cmt_1, the opening $\bar{\mathbf{w}}_1$ packed at 13 bits per coefficient into 4 160, 5 824 or 6 656 bytes, and the response $\mathbf{z}_1$ packed at 49 bits per coefficient into 12 544, 18 816 or 21 952 bytes, at levels I, III and V. The bodies received have the exact lengths of the broadcasts, which are the request with a message of 32 bytes, two bytes per index of T and the t first messages, the t openings, and the signature in the compact encoding described with the space on the node. Their content comes from the server, which does not change their time on the link, since TLS encrypts them. Each exchange adds 377 to 383 bytes of HTTP headers for its two requests, and the reference request adds 162 bytes.

The peers and the signature. Up to $t = 16$ all t signers run inside the job, and only the work of signer 1 counts in the latency. For $t = 128$ and $t = 1024$ the $t - 1$ peers send the first message and the opening of signer 1 under their own indices, as in our benchmarks of Section 7, which costs signer 1 exactly what t distinct peers cost, since its third round checks every opening against its first message and evaluates $t - 1$ masks. Such a session produces no signature of its own, so before the sessions the job draws one valid signature for each level and t with the sampler behind Table 4, which follows the exact distribution of the protocol, and delivers it in round three. Every signature delivered in the 231 sessions verified on the node.

The schedule. The job ran 30 sessions for $t = 2$ and for $t = 16$, 12 for $t = 128$ and 5 for $t = 1024$ at each level, 231 in total, between 09:52:38 and 09:57:20 UTC. It ran them in rounds over the 12 configurations, in a new random order in every round, and waited a random time of up to one second between sessions, 0.50 s at the median, so that every configuration spans the whole window and meets the link at every phase of the grid of 15 seconds.

Timing. The node measures durations with a monotonic clock of nanosecond resolution and stamps the start of every exchange with its system clock for the analysis of the phase. The latency of a session is the sum of the signing time of signer 1 in its three rounds and of the times of the three exchanges, that is the time from the start of round one until signer 1 holds the signature when the coordinator and the other signers answer at once. It excludes the reference request, the setup of the connection, which a signer that keeps its connection to the coordinator pays once, and the verification of the signature, which Table 14 reports separately.

Server time. The server reports in the Server-Timing header [74] of every response how long it worked on the request, 37 ms per exchange at the median, 174 ms at the 95th percentile and up to 2.5 s. A coordinator answers without this work, so we subtract it from every exchange and call the result the network-only time, which is the measure of Section 7.1, and Table 14 also gives the raw time. The same header reports the state of the TCP connection as the server sees it, including its smoothed round-trip time, its minimum round-trip time and its retransmitted segments, which the job records for every response.

I.5. Results of the Signing Sessions

All 231 sessions completed on their first attempt, with no reconnection and no HTTP error, all over IPv4 and TLS 1.3 with the cipher suite `TLS_AES_256_GCM_SHA384`, and a probe with the OpenSSL client of the same library negotiated X25519MLKEM768 with the server. Table 14 completes Table 9 for every configuration.

The server measured a smoothed round-trip time of 23.9 ms at the median over all responses, 16.6 ms at the 5th and 39.5 ms at the 95th percentile, with a minimum of 13.9 ms, and echo requests to a public resolver returned in 24.8 to 34.3 ms at the median (Table 16). The network-only time of the reference request, 33.0 to 38.9 ms at the median, exceeds the round trip of TCP, because it also contains work at both ends that the header does not report, so the network-only latency slightly overstates the cost of the network. The setup of a connection took 18.2 to 22.8 ms for TCP, one round trip, and 38.8 to 43.1 ms for the TLS handshake, whose flights carry the hybrid key shares and the certificate chain of the server. The server reported at least one retransmitted segment on 21 of the 231 connections, 148 segments in total.

Table 15 compares the median network-only latency with three predictions that use the signing time of the node from Table 17. The first is the model of Section 7, with round trips of 100 ms and the bytes sent at 100 Mbit/s, the second replaces its round trip with the network-only reference round trip of each configuration, and the third adds the bytes received to the second at the goodput of the openings at $t = 1024$ of each level, 115.6, 147.8 and 146.2 Mbit/s. The first prediction is too high for small t, since this link is faster than the model assumes, and too low at $t = 1024$, the second is too low everywhere, by up to 3.08 times at $t = 1024$, and the third comes within 10 to 17 percent at $t = 1024$, so the bytes that a signer receives decide its latency for large signer sets. For small t the measurement exceeds every prediction, because an exchange carries two requests and costs more than one round trip, the signing time inside a session exceeds that of the benchmark, as the time on the node below shows, and transfers of a few kilobytes do not reach the goodput of transfers of megabytes.

Reconfiguration instants. We say that a session crosses a reconfiguration instant when an instant falls between the

TABLE 14. ALL SIGNING SESSIONS, IN MILLISECONDS. TCP AND TLS ARE THE SETUP OF THE CONNECTION, REFERENCE THE EMPTY REQUEST RAW AND WITHOUT THE PROCESSING TIME OF THE SERVER (NET), RAW THE LATENCY OF A SESSION WITH THAT PROCESSING TIME, NETWORK ONLY THE LATENCY WITHOUT IT, VERIFY THE DECODING AND VERIFICATION OF THE SIGNATURE, VERIFIED THE RAW LATENCY UNTIL THE SIGNATURE IS VERIFIED, AND RECEIVED THE BYTES THAT THE SIGNER RECEIVES IN A SESSION. ALL VALUES EXCEPT THE 95TH PERCENTILES, THE MINIMA, THE MAXIMA AND RECEIVED ARE MEDIANS.

				Reference		Raw		Network only					
Level	t	TCP	TLS	raw	net	median	p95	min	median	max	Verify	Verified	Received
I	2	22.3	42.5	65.7	35.6	273.8	777.4	127.8	162.0	395.6	10.9	285.0	19 415
	16	19.3	42.8	65.8	38.6	316.6	614.3	133.1	176.4	298.1	10.9	327.9	79 250
	128	22.2	42.2	64.1	35.9	433.2	1112.0	218.5	294.3	666.1	11.0	444.3	554 997
	1024	19.2	39.1	64.9	38.9	659.0	1755.0	507.6	555.7	1843.1	10.6	669.5	4 356 584
III	2	20.8	42.7	65.8	38.1	311.1	814.0	154.9	190.9	225.5	14.1	325.0	28 179
	16	21.9	39.5	69.6	37.8	351.2	660.5	146.4	216.7	556.5	14.1	358.1	112 098
	128	21.1	38.8	68.8	36.8	422.9	562.0	263.3	308.6	349.2	14.2	436.1	778 954
	1024	18.2	39.1	61.7	35.2	808.9	874.9	639.1	648.6	762.7	14.1	820.7	6 107 665
V	2	22.3	43.1	68.8	38.2	345.8	1034.2	144.1	176.1	510.7	15.6	359.3	32 542
	16	22.8	42.7	70.5	37.9	353.6	816.5	166.4	213.6	286.7	15.7	369.6	128 724
	128	22.3	39.7	65.4	34.4	510.6	882.2	242.4	305.8	385.4	15.6	526.1	893 227
	1024	21.6	42.7	66.0	33.0	918.1	2112.3	632.7	724.1	2271.9	16.3	934.5	7 003 304

TABLE 15. MEDIAN NETWORK-ONLY LATENCY IN MILLISECONDS AGAINST THREE PREDICTIONS, WITH THE MEASUREMENT DIVIDED BY EACH PREDICTION IN PARENTHESES. MODEL IS THE MODEL OF SECTION 7, LINK RTT THE SAME MODEL WITH THE REFERENCE ROUND TRIP OF THE CONFIGURATION, AND RECEIVED THE LATTER WITH THE BYTES RECEIVED AT THE GOODPUT OF THE OPENINGS AT $t = 1024$ OF THE LEVEL. ALL USE THE SIGNING TIME OF THE NODE.

Level	t	Measured	Model	Link RTT	Received
I	2	162.0	305.0 (0.53)	112.0 (1.45)	113.1 (1.43)
	16	176.4	306.1 (0.58)	121.9 (1.45)	127.2 (1.39)
	128	294.3	315.2 (0.93)	122.8 (2.40)	161.0 (1.83)
	1024	555.7	387.9 (1.43)	204.7 (2.72)	505.9 (1.10)
III	2	190.9	306.8 (0.62)	121.2 (1.58)	122.0 (1.56)
	16	216.7	308.4 (0.70)	122.0 (1.78)	127.4 (1.70)
	128	308.6	322.1 (0.96)	132.3 (2.33)	173.9 (1.77)
	1024	648.6	422.2 (1.54)	227.8 (2.85)	557.7 (1.16)
V	2	176.1	307.8 (0.57)	122.4 (1.44)	123.5 (1.43)
	16	213.6	309.6 (0.69)	123.3 (1.73)	129.6 (1.65)
	128	305.8	323.7 (0.94)	127.0 (2.41)	175.2 (1.75)
	1024	724.1	436.2 (1.66)	235.3 (3.08)	617.9 (1.17)

start of its first round and the end of its third. A session lasts less than one second at the median, so 10 of the 231 sessions crossed one, and their network-only latency was 1.32 times the median of their configuration at the median and 1.46 times on average, against 1.00 and 1.05 for the other 221 sessions. The round-trip time itself did not depend on the phase, since the median smoothed round-trip time of the server lay between 22.4 and 25.2 ms in each of the 15 one-second bins of the time elapsed since the last instant. The two slowest sessions of the run, both at $t = 1024$, fall at reconfiguration instants. The level V session that started 2.7 s before the instant at second 27 crossed it and received its openings at 31.9 Mbit/s, against 126.7 to 178.8 Mbit/s in the four other sessions of its configuration, and the level I session that started 2.0 s after the instant at second 12 received its openings at 31.4 Mbit/s while the server measured a round trip of 108.3 ms, against 97.0 to 124.8 Mbit/s and 20.1 to 29.0 ms in the four other sessions of its configuration. These two sessions set the maxima and

TABLE 16. ICMP ECHO REQUESTS AT THE SIZES OF THE THIMZA OBJECTS, 20 PER SIZE, WITH THE PAYLOAD AND THE PACKET IN BYTES, THE REPLIES RECEIVED AND THEIR MEDIAN ROUND-TRIP TIME IN MILLISECONDS.

Payload	Packet	Object	Replies	RTT
56	84	reference	20/20	30.1
80	108	I round 1 commitment	20/20	27.6
120	148	III round 1 commitment	19/20	30.6
160	188	V round 1 commitment	20/20	24.8
1 024	1 052	reference	20/20	34.3
1 472	1 500	reference	20/20	30.6
3 872	3 900	I verification key	0/20	–
4 160	4 188	I round 2 opening	0/20	–
5 824	5 852	III round 2 opening	0/20	–
5 856	5 884	III verification key	0/20	–
6 656	6 684	V round 2 opening	0/20	–
7 200	7 228	V verification key	0/20	–
12 544	12 572	I round 3 response	0/20	–
12 742	12 770	I signature at $t = 1024$	0/20	–
18 816	18 844	III round 3 response	0/20	–
18 928	18 956	III signature at $t = 1024$	0/20	–
21 617	21 645	V signature at $t = 1024$	0/20	–
21 952	21 980	V round 3 response	0/20	–

the 95th percentiles of their rows, and the other 13 sessions at $t = 1024$ took between 507.6 and 787.9 ms.

I.6. Single Packets

The job sent 20 ICMP echo requests, at intervals of 0.2 s, to the public resolver at `8.8.8.8` for every payload size of Table 16, which are the sizes of the Thimza objects and three reference sizes, and it waited up to 3 s for each reply. A packet carries the payload, an IP header of 20 bytes and an ICMP header of 8 bytes. The path delivered every size whose packet fits the MTU of 1500 bytes, with one lost reply out of 120, and none of the 240 larger requests, which travel as IP fragments. On this path, therefore, only the first message of Thimza fits in one packet, and the openings, the responses, the keys and the signatures need a transport that splits them, such as TCP, which our sessions use, or QUIC.

TABLE 17. MEDIAN TIME OF THIMZA ON THE NODE IN MILLISECONDS, WITH THE M2 TIMES OF TABLE 8 IN PARENTHESES, AND THE COMBINER AND THE DEALER FOR $t = n = 1024$.

	Level I	Level III	Level V
Thimza, $t = 16$	4.8 (3.1)	6.5 (4.2)	7.3 (4.7)
Thimza, $t = 128$	13.8 (9.6)	20.1 (13.2)	21.4 (14.8)
Thimza, $t = 1024$	86.6 (60.2)	120.2 (83.9)	133.9 (95.3)
Verify	3.0	4.2	4.5
Combine	10.4	14.6	16.0
Key generation (s)	0.86	0.97	1.00

I.7. Time and Space on the Node

The time stage repeated our measurements of Thimza of Section 7 on the node with the same method, with 50 runs per point for t up to 256 and 25 runs from $t = 512$, and it reports medians, where Section 7 reports the best of three runs, and on the node the best run lies within 9 percent of the median at every point and within 2 percent at half of them. Table 17 gives the signing time of one signer at $t = 16$, 128 and 1024, with the values of Table 8 for the M2 in parentheses, and the time of the combiner, of the verifier and of key generation by the dealer for $n = 1024$. At $t = 1024$ the node is 1.40 to 1.44 times slower than the M2.

Inside a session the signer waits for the network between its rounds, and a computation that follows such a wait ran up to three times slower than in the loop of the time stage. The first round took 6.1 to 10.5 ms in the sessions against 2.0 to 3.7 ms in the loop, while the third round at $t = 1024$, which lasts about 0.1 s, took 1.00 to 1.13 times its time in the loop. This is consistent with the processor lowering its clock while the signer waits, since the node runs the `powersave` governor, and the latencies of Tables 9 and 14 include the slower computation, as a deployment on such a node would.

The space stage encoded every object of a signer at every level and $t \in \{2, 16, 128, 1024\}$, and drew signatures with the sampler behind Table 4, 30 at $t = 2$ and at $t = 16$, 15 at $t = 128$ and 8 at $t = 1024$ per level. The messages that a signer sends have exactly the sizes of Table 6 at every t, 16 784, 24 760 and 28 768 bytes in total at levels I, III and V, and the verification keys have 3 872, 5 856 and 7 200 bytes. Table 18 gives what a signer receives, the size of its token after round two, which it erases after round three, and the median size of a signature in a compact encoding that carries the 32-byte seed from which our implementation derives c and the coefficients of $\mathbf{z}$ and $\mathbf{h}$ in a lossless Golomb–Rice code. After round one the token has 16 821, 24 797 and 28 805 bytes at levels I, III and V for every t, and a signer receives 0.5, 3.8 to 4.0, 30.3 to 32.4 and 242.7 to 258.8 times as many bytes as it sends at $t = 2$, 16, 128 and 1024. All 249 signatures verified after decoding, the signatures of one row differ in size by at most 47 bytes, and every one is smaller than the size model of Table 6. The compact signature grows with t, since the width of $\mathbf{z}$ grows with $\sqrt{t}$, so a deployment that hides the size of the signer set pads it to its size at $T_{\max}$.

TABLE 18. WHAT ONE SIGNER RECEIVES AND KEEPS, IN BYTES, WHICH IS THE REQUEST AND THE OPENINGS, ITS TOKEN AFTER ROUND TWO, AND THE MEDIAN SIZE OF A COMPACT SIGNATURE NEXT TO THE SIZE MODEL OF TABLE 6.

		Received			Signature	
Level	t	Request	Openings	Token	compact	model
I	2	200	8 320	16 907	10 898	13 152
	16	1 348	66 560	16 935	11 345	13 152
	128	10 532	532 480	17 159	11 973	13 152
	1024	84 004	4 259 840	18 951	12 735	13 152
III	2	280	11 648	24 923	16 248	19 568
	16	1 988	93 184	24 951	16 913	19 568
	128	15 652	745 472	25 175	17 837	19 568
	1024	124 964	5 963 776	26 967	18 936	19 568
V	2	360	13 312	28 971	18 881	22 272
	16	2 628	106 496	28 999	19 594	22 272
	128	20 772	851 968	29 223	20 477	22 272
	1024	165 924	6 815 744	31 015	21 639	22 272

I.8. Scope of the Measurement

The job measured one node, one constellation, one server and a window of 4.7 minutes that spans about 19 intervals of the grid of 15 seconds, so its latencies describe this path on this day, while the method applies unchanged to any node of the testbed and to any server that offers the two endpoints. Only signer 1 crosses the satellite link, and the coordinator and the other signers stand for parties with fast connections, so the latency is that of a signer behind a satellite link, and a coordinator behind such a link would in addition receive the t messages of every round. Peers above $t = 16$ are simulated as in our benchmarks, which leaves the work and the bytes of signer 1 unchanged. The broadcasts have the exact sizes but not the contents of real broadcasts, which TLS makes irrelevant for their time on the link. The server is a speed test rather than a coordinator, so we remove the processing time that it reports and give the raw latency as well. The implementation is the prototype of Section 7 in Python, which is not constant time, so its signing times are those of this prototype. The node shares its connection with the institution or household that hosts it, and other traffic on that connection is part of the conditions of the run, as it is of a real deployment. The analysis of the phase uses the system clock of the node, which the container shares with the host, and an offset of that clock from UTC would shift the instants, which the job cannot measure from inside the container.

The job sent 5.41 MB and received 126.07 MB in its sessions and sent 2.87 MB of echo requests, it used the speed test only through its public endpoints, on average one request every 174 ms, and it collected no address, no location and no personal data.

Appendix J. The `EasyCrypt` Development

Table 19 describes the 50 files of the development in dependency order, with the content and the main lemmas of each group of files. The first groups formalize the ring, the sums, the sharing and the masks, the middle groups

the protocol, the notions, the hierarchy and the theorems of Sections 3 and 4, and the last groups the oracle theories, the game chain of Theorem 8 from `ChainGame` to `ChainFin`, the bounds of its abort events, and `ChainMain`, which composes them into the lemma `thimza_main` of Figure 6. The script `check.sh` removes all cached results, scans for admitted steps, checks every file, and fails if an axiom appears outside three places. These are the oracle theories `QROM` and `ExtRO`, whose 10 axioms are the quantum facts and their parameters, the file `ChainBase`, whose 12 axioms state that abstract distributions are lossless or uniform and that messages and digest inputs form finite types, and the hypotheses of the sections of the game chain, namely termination and query bounds of the adversary, an injective encoding of the inputs of H_1, the regularity of openings and of aggregate commitments, and evaluation points that are units with unit differences, which become premises of the closed lemmas, as does the rounding bound `rounding_fact` of the final step. Every clone of a library theory ends with `proof *`, and the six clones of the oracle theories keep exactly the quantum facts.

Ethics Considerations. The replay attack of Theorem 3 lets an adversary that corrupts $t - 1$ parties recover the signing key from a stateless or snapshot-restored signer, but not from signers that keep their tokens as specified, in memory that is not snapshotted or behind a monotonic counter (Section 8). We judged that the benefit of practical post-quantum threshold signatures, and of warning implementers before deployment, outweighs this risk. The LEOScope experiment did not record any personal data.

AI Usage Policy. Grammarly's AI assisted in editing text, and Claude in implementing experiments.

TABLE 19. THE FILES OF THE EASYCRYPT DEVELOPMENT AND THEIR MAIN RESULTS.

File	Content	Main lemmas
`Params`	the ring $\mathbb{Z}_q[X]/(X^\varphi+1)$, vectors and matrices as subtypes, the matrix $\mathbf{A}$	clone obligations, `mulAD`, `mulAZ`
`Types`, `BigOps`, `PolyList`	committees, sums and products over lists, polynomials as lists	`vmap_cat`, `rmap_perm`, `peval_cons`
`Lagrange`, `Sharing`, `ShamirLagrange`	Lagrange coefficients and Shamir sharing over $\mathcal{R}_q$	`shr_recons`, `shr_hiding`, `lam_of_recons`
`Masking`, `MaskAlgebra`	antisymmetric pair masks and their sums	`mask_sum_zero`, `mask_class_sum`
`ViewPartition`, `Corruption`	masks under partitions of the signer set by views, honest and corrupt members	`view_partition`, `honest_corrupted_split`
`Simulator`, `ClassSim`	one time pads of masked responses, simulation of the responses of a class	`class_sim`, `class_prefix`
`Statelessness`	the replay attack	`replay_recovers_share`
`QROM`	oracles drawn with `dfun`, the counting wrapper, collision, adaptive reprogramming, lazily sampled first messages	`dfh_reprog`, `dfh_split`, axioms `pr_col`, `ghhm`, `o2h_lazy`
`Protocol`	tokens, the three rounds with their checks and erasure, combining, verification	`correctness`, `hint_correct`, `erase_dead`
`Notions`, `Games`, `Hierarchy`	the sets S_1 to S_4, the conditions tf_0 to tf_4, the game with its oracles and logs	`game_inv`, `uf_hierarchy`, `uf4_o_r`
`Privacy`, `Interchange`	honest executions, the Raccoon verifier and signer	`signer_set_privacy`, `single_signer`
`HintAlg`, `Extract`	the hint embedding, extraction of a SelfTargetMSIS solution from a forgery	`hint_commit`, `honest_aggregate`, `forgery_solves`
`Assumptions`, `Security`	games of PRF, HintMLWE and SelfTargetMSIS, the final reduction	`final_reduction`
`ExtRO`, `FinEnum`	the extractable simulator of [15], finite enumerations of oracle domains	axioms `ext_sim`, `ext_sound`; `enum_pair`
`ChainBase`, `ChainGame`	oracle instances, adversary interface with query counters, the game G_0 and the hop to random pair functions	`dseedf_split`, `hop_prf`
`ChainCol`	collisions of digests and of first messages in G_1	`col1_bound`, `col0_bound`
`ChainSim`	responses without honest pair values, up to digest collisions	`fresh_key`, `o3_sim`, `run_sim`, `g1_g1s`
`ChainDead`, `ChainLazy`	dropping unused pair values, lazily sampled first messages	`g1s_g1t`, `g1t_lazy0`, `bl_cnt`, `g1t_lazy`
`ChainFlag`, `ChainGA`	the flags b_1, b_2, b_3 of an accepted check against an inconsistent token, a repeated first message and a response to other first messages with the same digest	`g1_ga`, `ga_split`, `ga_gaf`, `gaf_gag`
`ChainExt`	online extraction of corrupt first messages	`ga_ext`
`ChainRep`, `ChainProg`	reprogramming of H_c at complete requests, explicit programming	`ge_rep`, `rep_gp`
`ChainHint`, `ChainCons`	responses from hints up to the flag b_H, which the consistency of the run covers by a point programmed twice or a completion beyond the budget	`gp_gh`, `gp_cons`, `gp_split`
`ShamirHide`, `ChainFin`	uniform corrupt shares, the games of HintMLWE and SelfTargetMSIS as $\mathcal{B}_2$ and $\mathcal{B}_3$ run them	`shr_shift`, `gh_hm`, `hm_stm`, `gh_bound`
`ChainPB`, `ChainB2`, `ChainB3`	the flags of programmed points and completions, of repeated first messages and of repeated digests	`gp_bp`, `gp_bc_pr`, `ga_b2`, `ga_b3`
`ChainB1a` to `ChainB1d`	the flag b_1: transport to eager commitments, a collision there, and a logged opening point by the failure event lemma	`bbx_o2h`, `bbx_b1col`, `lef_badE`, `ga_b1`
`ChainMain`	Theorem 8 as one closed lemma	`thimza_main`